

The group $G(\mathbf{m})$ is perfect: a machine-checked proof

Abstract

We present a formal verification of Theorem 3.3 of the source. The theorem states that the group $G(\mathbf{m})$ is perfect, where $G(\mathbf{m})$ is given by generators $H_1(s), H_2(s), X_{-1}(u), Y_{-1}(u), X_{\ell,jk}(u), Y_{\ell,jk}(u)$ and the relations (H), (Re), (Im), (U). In the formalization the coefficient function c of the modular function J is replaced by an arbitrary function $c : \mathbb{N} \rightarrow \mathbb{N}$, and the only hypothesis is $c(2) \geq 1$. The group is built as a presented group, the free group modulo the normal closure of the relators. The formal theorem states that its commutator subgroup is the whole group. The proof follows the source: it shows that each generator is a product of commutators, using (U:3a), (U:3b), (Im:1), (Im:6b), (Im:6d), (Re:3), (Im:3), (H:1a) and (Re:5a).

Source	Carbone et al., G-Simple (draft), GSimple_Theorem_3_3
Lean	leanprover/lean4:v4.23.0, Mathlib 37df177aaa
Machine checks	compiles; no sorry; statement matches target; module build
Axioms	propext, Classical.choice, Quot.sound
Independent review	FAITHFUL (claude-opus-5-5)
Agents	captain claude-opus-5-5; solvers aiprover; report claude-opus-5-5
Run	srv_20261004_123235_GSimple_Theorem_3_3

Contents

1	Statement	1
2	Formalization	3
3	Proof	8
3.1	Overview	8
3.2	Proof of the theorem	24
4	Independent review	25
A	Complete Lean file	26

1 Statement

Let $c(j)$ be the coefficient of q^j in the modular function

$$J(q) = \sum_{j \geq -1} c(j)q^j = \frac{1}{q} + 196884q + 21493760q^2 + 864299970q^3 + \dots,$$

so that $c(-1) = 1$, $c(0) = 0$ and $c(1) = 196884$. Define the index set

$$E = \{(\ell, j, k) \mid j \in \mathbb{N}, 1 \leq k \leq c(j), 0 \leq \ell < j\}.$$

For $0 \leq \ell < j$ put

$$c_{\ell j} := (-1)^{\ell+1} \binom{j-1}{\ell} (\ell+1)(j-\ell) \in \mathbb{C}.$$

Let $\mathcal{X}$ be the set of symbols $H_1(s)$, $H_2(s)$, $X_{-1}(u)$, $Y_{-1}(u)$, $X_{\ell,jk}(u)$, $Y_{\ell,jk}(u)$, where $s \in \mathbb{C}^\times$, $u \in \mathbb{C}$ and $(\ell, j, k) \in E$. Let $F(\mathcal{X})$ be the free group on $\mathcal{X}$. The commutator is $(a, b) := aba^{-1}b^{-1}$.

The relations $\mathcal{R}$ below are imposed for all $s, t \in \mathbb{C}^\times$, $u, v \in \mathbb{C}$ and $(\ell, j, k), (m, p, q) \in E$.

$$\begin{aligned}
\tilde{w}_{-1}(s) &:= X_{-1}(s)Y_{-1}(-s^{-1})X_{-1}(s), & \tilde{w}_{-1} &:= \tilde{w}_{-1}(1), & (\text{Re:0}) \\
\tilde{w}_{\ell,jk}(s) &:= X_{\ell,jk}(s)Y_{\ell,jk}\left(\frac{-s^{-1}}{c_{\ell j}}\right)X_{\ell,jk}(s), & \tilde{w}_{\ell,jk} &:= \tilde{w}_{\ell,jk}(1), & (\text{Im:0}) \\
H_1(s)H_1(t) &= H_1(st), & & & (\text{H:1a}) \\
H_2(s)H_2(t) &= H_2(st), & & & (\text{H:1b}) \\
H_1(s)H_2(t) &= H_2(t)H_1(s), & & & (\text{H:2}) \\
X_{-1}(u)X_{-1}(v) &= X_{-1}(u+v), & & & (\text{Re:1a}) \\
Y_{-1}(u)Y_{-1}(v) &= Y_{-1}(u+v), & & & (\text{Re:1b}) \\
Y_{-1}(-t)X_{-1}(s)Y_{-1}(t) &= X_{-1}(-t^{-1})Y_{-1}(-t^2s)X_{-1}(t^{-1}), & & & (\text{Re:2}) \\
\tilde{w}_{-1}(s)\tilde{w}_{-1} &= H_1(-s)H_2(-s^{-1}), & & & (\text{Re:3}) \\
\tilde{w}_{-1}X_{-1}(u)\tilde{w}_{-1}^{-1} &= Y_{-1}(-u), & & & (\text{Re:4a}) \\
\tilde{w}_{-1}Y_{-1}(u)\tilde{w}_{-1}^{-1} &= X_{-1}(-u), & & & (\text{Re:4b}) \\
\tilde{w}_{-1}H_1(s)\tilde{w}_{-1}^{-1} &= H_2(s), & & & (\text{Re:5a}) \\
\tilde{w}_{-1}H_2(s)\tilde{w}_{-1}^{-1} &= H_1(s), & & & (\text{Re:5b}) \\
H_1(s)X_{-1}(u)H_1(s)^{-1} &= X_{-1}(su), & & & (\text{Re:6a}) \\
H_2(s)X_{-1}(u)H_2(s)^{-1} &= X_{-1}(s^{-1}u), & & & (\text{Re:6b}) \\
H_1(s)Y_{-1}(u)H_1(s)^{-1} &= Y_{-1}(s^{-1}u), & & & (\text{Re:6c}) \\
H_2(s)Y_{-1}(u)H_2(s)^{-1} &= Y_{-1}(su), & & & (\text{Re:6d}) \\
X_{\ell,jk}(u)X_{\ell,jk}(v) &= X_{\ell,jk}(u+v), & & & (\text{Im:1a}) \\
Y_{\ell,jk}(u)Y_{\ell,jk}(v) &= Y_{\ell,jk}(u+v), & & & (\text{Im:1b}) \\
Y_{\ell,jk}(-t)X_{\ell,jk}(s)Y_{\ell,jk}(t) &= X_{\ell,jk}\left(\frac{-t^{-1}}{c_{\ell j}}\right)Y_{\ell,jk}(-c_{\ell j}t^2s)X_{\ell,jk}\left(\frac{t^{-1}}{c_{\ell j}}\right), & & & (\text{Im:2}) \\
\tilde{w}_{\ell,jk}(s^{(\ell+1)(j-\ell)})\tilde{w}_{\ell,jk} &= H_1((-s)^{j-\ell})H_2((-s)^{\ell+1}), & & & (\text{Im:3}) \\
\tilde{w}_{\ell,jk}X_{\ell,jk}(u)\tilde{w}_{\ell,jk}^{-1} &= Y_{\ell,jk}(-u/c_{\ell j}), & & & (\text{Im:4a}) \\
\tilde{w}_{\ell,jk}Y_{\ell,jk}(u)\tilde{w}_{\ell,jk}^{-1} &= X_{\ell,jk}(-c_{\ell j}u), & & & (\text{Im:4b}) \\
\tilde{w}_{\ell,jk}H_1(s^{j-\ell})\tilde{w}_{\ell,jk}^{-1} &= H_2(s^{-(\ell+1)}), & & & (\text{Im:5a}) \\
\tilde{w}_{\ell,jk}H_2(s^{\ell+1})\tilde{w}_{\ell,jk}^{-1} &= H_1(s^{-(j-\ell)}), & & & (\text{Im:5b}) \\
H_1(s)X_{\ell,jk}(u)H_1(s)^{-1} &= X_{\ell,jk}(s^{\ell+1}u), & & & (\text{Im:6a}) \\
H_2(s)X_{\ell,jk}(u)H_2(s)^{-1} &= X_{\ell,jk}(s^{j-\ell}u), & & & (\text{Im:6b}) \\
H_1(s)Y_{\ell,jk}(u)H_1(s)^{-1} &= Y_{\ell,jk}(s^{-(\ell+1)}u), & & & (\text{Im:6c}) \\
H_2(s)Y_{\ell,jk}(u)H_2(s)^{-1} &= Y_{\ell,jk}(s^{-(j-\ell)}u), & & & (\text{Im:6d}) \\
(X_{-1}(u), X_{j-1,jk}(v)) &= 1, & & & (\text{U:1a}) \\
(Y_{-1}(u), Y_{j-1,jk}(v)) &= 1, & & & (\text{U:1b}) \\
(Y_{-1}(u), X_{0,jk}(v)) &= 1, & & & (\text{U:1c}) \\
(X_{-1}(u), Y_{0,jk}(v)) &= 1, & & & (\text{U:1d}) \\
(X_{\ell,jk}(u), Y_{m,pq}(v)) &= 1 \quad \text{if } j \neq p \text{ or } k \neq q \text{ or } |\ell - m| > 1, & & & (\text{U:2}) \\
(X_{1,2k}(u), Y_{0,2k}(v)) &= X_{-1}(uv), & & & (\text{U:3a}) \\
(X_{0,2k}(u), Y_{1,2k}(v)) &= Y_{-1}(uv), & & & (\text{U:3b}) \\
\tilde{w}_{-1}X_{\ell,jk}(u)\tilde{w}_{-1}^{-1} &= X_{j-1-\ell,jk}((-1)^{j-\ell-1}u), & & & (\text{U:4a}) \\
\tilde{w}_{-1}Y_{\ell,jk}(u)\tilde{w}_{-1}^{-1} &= Y_{j-1-\ell,jk}((-1)^{j-\ell-1}u). & & & (\text{U:4b})
\end{aligned}$$

Define $G(\mathbf{m}) = \langle \mathcal{X} \mid \mathcal{R} \rangle = F(\mathcal{X})/N_{\mathcal{R}}$, where $N_{\mathcal{R}}$ is the normal closure of $\mathcal{R}$.

Theorem 1.1 (Theorem 3.3 of the source). *The group $G(\mathbf{m})$ is perfect, that is, $G(\mathbf{m}) = (G(\mathbf{m}), G(\mathbf{m}))$.*

2 Formalization

Throughout, $c : \mathbb{N} \rightarrow \mathbb{N}$ is an arbitrary function and $\mathbb{N} = \{0, 1, 2, \dots\}$.

Definition 2.1 (Index set). For $\ell, j, k \in \mathbb{N}$, say that (ℓ, j, k) is *valid* if $\ell < j$, $1 \leq k$ and $k \leq c(j)$. The set of valid triples is exactly the set E of the source, built from the given c . Since $\ell < j$ forces $j \geq 1$, the values $c(-1)$ and $c(0)$ play no role.

Lean: Valid

```

| /-- Index set E:  $(\ell, j, k)$  with  $0 \leq \ell < j$  and  $1 \leq k \leq c(j)$ . -/
| def Valid (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (l j k :  $\mathbb{N}$ ) : Prop := l < j  $\wedge$  1  $\leq$  k  $\wedge$  k  $\leq$  c j

```

Definition 2.2 (Generators). The set $\mathcal{X}_c$ consists of the symbols $H_1(s)$ and $H_2(s)$ for $s \in \mathbb{C}^\times$, $X_{-1}(u)$ and $Y_{-1}(u)$ for $u \in \mathbb{C}$, and $X_{\ell, jk}(u)$ and $Y_{\ell, jk}(u)$ for $u \in \mathbb{C}$ and valid (ℓ, j, k) .

Formally, the last two kinds carry a proof of validity. By proof irrelevance, any two such proofs give the same symbol, so $\mathcal{X}_c$ is in natural bijection with the source's $\mathcal{X}$.

Lean: Gen

```

| /-- The generating symbols. -/
| inductive Gen (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Type
|   H1 :  $\mathbb{C}^\times \rightarrow$  Gen c
|   H2 :  $\mathbb{C}^\times \rightarrow$  Gen c
|   Xm :  $\mathbb{C} \rightarrow$  Gen c
|   Ym :  $\mathbb{C} \rightarrow$  Gen c
|   X (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : Gen c
|   Y (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : Gen c

```

Definition 2.3. $F_c := F(\mathcal{X}_c)$ is the free group on $\mathcal{X}_c$.

Lean: F

```

| abbrev F (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) := FreeGroup (Gen c)

```

Definition 2.4. The following maps send a parameter to the corresponding free generator of F_c :

- $s \mapsto H_1(s)$ for $s \in \mathbb{C}^\times$;
- $s \mapsto H_2(s)$ for $s \in \mathbb{C}^\times$;
- $u \mapsto X_{-1}(u)$ for $u \in \mathbb{C}$;
- $u \mapsto Y_{-1}(u)$ for $u \in \mathbb{C}$;
- $u \mapsto X_{\ell, jk}(u)$ for valid (ℓ, j, k) and $u \in \mathbb{C}$;
- $u \mapsto Y_{\ell, jk}(u)$ for valid (ℓ, j, k) and $u \in \mathbb{C}$.

Lean: h1

```
| def h1 {c : ℕ → ℕ} (s : ℂ×) : F c := FreeGroup.of (Gen.H1 s)
```

Lean: h2

```
| def h2 {c : ℕ → ℕ} (s : ℂ×) : F c := FreeGroup.of (Gen.H2 s)
```

Lean: xm

```
| def xm {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Xm u)
```

Lean: ym

```
| def ym {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Ym u)
```

Lean: xx

```
| def xx {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.X l j k h u)
```

Lean: yy

```
| def yy {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.Y l j k h u)
```

Definition 2.5. For $\ell, j \in \mathbb{N}$,

$$c_{\ell j} := (-1)^{\ell+1} \binom{j \dot{-} 1}{\ell} (\ell + 1)(j \dot{-} \ell) \in \mathbb{C},$$

where $\dot{-}$ is truncated subtraction on $\mathbb{N}$. For valid triples we have $\ell < j$, so this agrees with the source's $c_{\ell j}$. In particular $c_{\ell j} \neq 0$ there.

Lean: cc

```
| /-- c_{\ell j} = (-1)^{(\ell+1)} * binom(j-1, \ell) * (\ell+1) * (j-\ell). -/
noncomputable def cc (l j : ℕ) : ℂ :=
  (-1 : ℂ) ^ (l + 1) * ((j - 1).choose l : ℕ) : ℂ * ((l : ℂ) + 1) * ((j - l : ℕ) : ℂ)
```

Definition 2.6. For $a, b \in F_c$:

- the commutator is $(a, b) := aba^{-1}b^{-1}$;
- the relator of the equation $a = b$ is $\rho(a, b) := ab^{-1}$.

Lean: cm

```
| /-- commutator (a,b) = a b a-1 b-1 -/
| def cm {c : ℕ → ℕ} (a b : F c) : F c := a * b * a-1 * b-1
```

Lean: eqr

```
| /-- relator for the equation a = b -/
| def eqr {c : ℕ → ℕ} (a b : F c) : F c := a * b-1
```

Definition 2.7. For $s \in \mathbb{C}^\times$ and valid (ℓ, j, k) , define elements of F_c :

$$\tilde{w}_{-1}(s) := X_{-1}(s) Y_{-1}(-s^{-1}) X_{-1}(s), \quad \tilde{w}_{\ell,jk}(s) := X_{\ell,jk}(s) Y_{\ell,jk}(-s^{-1}/c_{\ell j}) X_{\ell,jk}(s).$$

These are (Re:0) and (Im:0). Write $\tilde{w}_{-1} := \tilde{w}_{-1}(1)$ and $\tilde{w}_{\ell,jk} := \tilde{w}_{\ell,jk}(1)$.

Lean: wm

```
| noncomputable def wm {c : ℕ → ℕ} (s : ℂ×) : F c :=
|   xm (s : ℂ) * ym (-(s : ℂ)-1) * xm (s : ℂ)
```

Lean: ww

```
| noncomputable def ww {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) : F c :=
|   xx l j k h (s : ℂ) * yy l j k h (-(s : ℂ)-1) / cc l j * xx l j k h (s : ℂ)
```

Definition 2.8 (Relators). $\mathcal{R}_c \subseteq F_c$ is the set of the following elements.

- For every relation of the source written as an equation $a = b$, the relator $\rho(a, b)$. These are (H:1a), (H:1b), (H:2), (Re:1a)–(Re:6d), (Im:1a)–(Im:6d), (U:3a), (U:3b), (U:4a), (U:4b).
- For every relation of the form $(a, b) = 1$, the relator (a, b) . These are (U:1a)–(U:1d) and (U:2).

The parameters range exactly as in the source: $s, t \in \mathbb{C}^\times$, $u, v \in \mathbb{C}$, all triples valid. Further remarks:

- In (U:1a) and (U:1b) the triple $(j-1, j, k)$ is required to be valid; in (U:1c) and (U:1d), the triple $(0, j, k)$.
- In (U:2) the condition reads $j \neq p$ or $k \neq q$ or $|\ell - m| > 1$, with $|\ell - m|$ computed in $\mathbb{Z}$.
- In (U:3a) and (U:3b) both $(0, 2, k)$ and $(1, 2, k)$ must be valid.
- In (U:4a) and (U:4b) the target triple $(j-1-\ell, j, k)$ is also required to be valid. This is automatic for valid (ℓ, j, k) .
- Negative exponents are written as inverses of positive powers, e.g. $H_2((s^{\ell+1})^{-1})$ in (Im:5a). This is the same element.

Each relator is written out explicitly in F_c , with $c_{\ell j}$, $\tilde{w}_{-1}$ and $\tilde{w}_{\ell,jk}$ as above. A check relation by relation shows that $\mathcal{R}_c$ is exactly the source's $\mathcal{R}$ (built from c).

Lean: Rel

```

/-- The defining relators. -/
inductive Rel (c : ℕ → ℕ) : F c → Prop
| H1a (s t : ℂx) : Rel c (eqv (h1 s * h1 t) (h1 (s * t)))
| H1b (s t : ℂx) : Rel c (eqv (h2 s * h2 t) (h2 (s * t)))
| H2 (s t : ℂx) : Rel c (eqv (h1 s * h2 t) (h2 t * h1 s))
| Re1a (u v : ℂ) : Rel c (eqv (xm u * xm v) (xm (u + v)))
| Re1b (u v : ℂ) : Rel c (eqv (ym u * ym v) (ym (u + v)))
| Re2 (s t : ℂx) : Rel c (eqv (ym (-(t : ℂ)) * xm (s : ℂ) * ym (t : ℂ))
  (xm (-(t : ℂ)-1) * ym (-(t : ℂ) ^ 2 * (s : ℂ)) * xm ((t : ℂ)-1)))
| Re3 (s : ℂx) : Rel c (eqv (wm s * wm 1) (h1 (-s) * h2 (-(s-1))))
| Re4a (u : ℂ) : Rel c (eqv (wm 1 * xm u * (wm 1)-1) (ym (-u)))
| Re4b (u : ℂ) : Rel c (eqv (wm 1 * ym u * (wm 1)-1) (xm (-u)))
| Re5a (s : ℂx) : Rel c (eqv (wm 1 * h1 s * (wm 1)-1) (h2 s))
| Re5b (s : ℂx) : Rel c (eqv (wm 1 * h2 s * (wm 1)-1) (h1 s))
| Re6a (s : ℂx) (u : ℂ) : Rel c (eqv (h1 s * xm u * (h1 s)-1) (xm ((s : ℂ) * u)))
| Re6b (s : ℂx) (u : ℂ) : Rel c (eqv (h2 s * xm u * (h2 s)-1) (xm ((s : ℂ)-1 * u)))
| Re6c (s : ℂx) (u : ℂ) : Rel c (eqv (h1 s * ym u * (h1 s)-1) (ym ((s : ℂ)-1 * u)))
| Re6d (s : ℂx) (u : ℂ) : Rel c (eqv (h2 s * ym u * (h2 s)-1) (ym ((s : ℂ) * u)))
| Im1a (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqv (xx l j k h u * xx l j k h v) (xx l j k h (u + v)))
| Im1b (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqv (yy l j k h u * yy l j k h v) (yy l j k h (u + v)))
| Im2 (l j k : ℕ) (h : Valid c l j k) (s t : ℂx) :
  Rel c (eqv (yy l j k h (-(t : ℂ)) * xx l j k h (s : ℂ) * yy l j k h (t : ℂ))
    (xx l j k h (-(t : ℂ)-1) / cc l j) *
    yy l j k h (-(cc l j) * (t : ℂ) ^ 2 * (s : ℂ)) *
    xx l j k h ((t : ℂ)-1 / cc l j)))
| Im3 (l j k : ℕ) (h : Valid c l j k) (s : ℂx) :
  Rel c (eqv (ww l j k h (s ^ ((l + 1) * (j - 1))) * ww l j k h 1)
    (h1 (-(s) ^ (j - 1)) * h2 (-(s) ^ (l + 1))))
| Im4a (l j k : ℕ) (h : Valid c l j k) (u : ℂ) :
  Rel c (eqv (ww l j k h 1 * xx l j k h u * (ww l j k h 1)-1) (yy l j k h (-u / cc l j)))
| Im4b (l j k : ℕ) (h : Valid c l j k) (u : ℂ) :
  Rel c (eqv (ww l j k h 1 * yy l j k h u * (ww l j k h 1)-1) (xx l j k h (-(cc l j) * u)))
| Im5a (l j k : ℕ) (h : Valid c l j k) (s : ℂx) :
  Rel c (eqv (ww l j k h 1 * h1 (s ^ (j - 1)) * (ww l j k h 1)-1) (h2 ((s ^ (l + 1))-1)))
| Im5b (l j k : ℕ) (h : Valid c l j k) (s : ℂx) :
  Rel c (eqv (ww l j k h 1 * h2 (s ^ (l + 1)) * (ww l j k h 1)-1) (h1 ((s ^ (j - 1))-1)))
| Im6a (l j k : ℕ) (h : Valid c l j k) (s : ℂx) (u : ℂ) :
  Rel c (eqv (h1 s * xx l j k h u * (h1 s)-1) (xx l j k h ((s : ℂ) ^ (l + 1) * u)))
| Im6b (l j k : ℕ) (h : Valid c l j k) (s : ℂx) (u : ℂ) :
  Rel c (eqv (h2 s * xx l j k h u * (h2 s)-1) (xx l j k h ((s : ℂ) ^ (j - 1) * u)))
| Im6c (l j k : ℕ) (h : Valid c l j k) (s : ℂx) (u : ℂ) :
  Rel c (eqv (h1 s * yy l j k h u * (h1 s)-1) (yy l j k h (((s : ℂ) ^ (l + 1))-1 * u)))
| Im6d (l j k : ℕ) (h : Valid c l j k) (s : ℂx) (u : ℂ) :
  Rel c (eqv (h2 s * yy l j k h u * (h2 s)-1) (yy l j k h (((s : ℂ) ^ (j - 1))-1 * u)))
| U1a (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (xm u) (xx (j - 1) j k h v))
| U1b (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (ym u) (yy (j - 1) j k h v))
| U1c (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (ym u) (xx 0 j k h v))
| U1d (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (xm u) (yy 0 j k h v))
| U2 (l j k : ℕ) (h : Valid c l j k) (m p q : ℕ) (h' : Valid c m p q) (u v : ℂ) :
  (hne : j ≠ p ∨ k ≠ q ∨ 1 < Int.natAbs ((l : ℤ) - (m : ℤ))) :
  Rel c (cm (xx l j k h u) (yy m p q h' v))

```

```

| U3a (k : ℕ) (h : Valid c 1 2 k) (h' : Valid c 0 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 1 2 k h u) (yy 0 2 k h' v)) (xm (u * v)))
| U3b (k : ℕ) (h : Valid c 0 2 k) (h' : Valid c 1 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 0 2 k h u) (yy 1 2 k h' v)) (ym (u * v)))
| U4a (l j k : ℕ) (h : Valid c l j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * xx l j k h u * (wm 1)-1)
    (xx (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))
| U4b (l j k : ℕ) (h : Valid c l j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * yy l j k h u * (wm 1)-1)
    (yy (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))

```

Lean: Rels

```

def Rels (c : ℕ → ℕ) : Set (F c) := {r | Rel c r}

```

Definition 2.9. $G_c := F_c/N_{\mathcal{R}_c}$, where $N_{\mathcal{R}_c}$ is the normal closure of $\mathcal{R}_c$. Let $\pi : F_c \rightarrow G_c$ be the quotient homomorphism.

Lean: Grp

```

/-- The group G(m) = ⟨X | R⟩. -/
abbrev Grp (c : ℕ → ℕ) : Type := PresentedGroup (Rels c)

```

The target statement. For a group G , (G, G) denotes the commutator subgroup, i.e. the subgroup generated by all $(g, h) = ghg^{-1}h^{-1}$. This is the same convention as in the source.

Lean: target statement gsimple_g_m_perfect, fixed before the proof; solution is checked to have exactly this type

```

theorem gsimple_g_m_perfect (c : ℕ → ℕ) (hc : 1 ≤ c 2) :
  commutator (GM.Grp c) = 1

```

The target statement, proved as solution below, says:

For every $c : \mathbb{N} \rightarrow \mathbb{N}$ with $c(2) \geq 1$, $(G_c, G_c) = G_c$.

Relation to the source:

- The source's group $G(\mathbf{m})$ is G_c for c equal to the coefficients of J restricted to $\mathbb{N}$. That function satisfies $c(2) = 21493760 \geq 1$.
- The modular function J is not formalized. Accordingly, the value $c(2) = 21493760$ is a fact from the source, not something checked formally.
- The formal theorem is more general than the source: it holds for every c with $c(2) \geq 1$. As the problem set notes, the proof uses c only through E and through the existence of $(0, 2, k), (1, 2, k) \in E$ for some k . The formal proof takes $k = 1$.

3 Proof

3.1 Overview

The argument shows that (G_c, G_c) contains the image of every generator, so it equals G_c . The lemmas map to the steps of the source proof as follows.

- Bookkeeping in the presented group: [Lemmas 3.1 to 3.4](#).
- X_{-1}, Y_{-1} via (U:3a) and (U:3b): [Lemmas 3.5 and 3.6](#).
- $X_{\ell,jk}$ via (Im:1a) and (Im:6b): [Lemmas 3.8, 3.9, 3.11, 3.13 and 3.16](#).
- $Y_{\ell,jk}$ via (Im:1b) and (Im:6d): [Lemmas 3.10, 3.12, 3.14, 3.15 and 3.17](#).
- The elements $\tilde{w}$: [Lemmas 3.7 and 3.18](#).
- H_1 via (Re:3), (Im:3) with $(\ell, j) = (0, 2)$, and (H:1a): [Lemmas 3.19 to 3.25](#).
- H_2 via (Re:5a): [Lemmas 3.26 and 3.27](#).

The formal proof differs from the source in the following points.

1. $X_{-1}(u)$ and $Y_{-1}(u)$. Instead of factoring $z = uv$, the proof uses (U:3a) and (U:3b) with $v = 1$ and $k = 1$. The source's step for Y_{-1} contains a typographical slip (" $Y_{-1}(uv)(X_{0,2k}(u), Y_{1,2k}(v))$ "); the formal step uses the relation (U:3b) as stated.
2. *Choice of s* . The element s with $s^{j-\ell} \neq 1$ is taken to be $s = 2$. For $Y_{\ell,jk}$ the multiplier is $(s^{j-\ell})^{-1} \neq 1$, exactly as in the source.
3. *The step $H_1(z^3)$* . The source uses $H_1(z^{-2})^{-1} = H_1(z^2)$ and $H_1(z)H_1(z^2) = H_1(z^3)$. Formally, (H:1a) is first used to show that $s \mapsto \pi(H_1(s))$ is a homomorphism $\mathbb{C}^\times \rightarrow G_c$. The H_2 -factors cancel by group arithmetic alone; (H:2) is not needed.
4. *The step $H_2(s)$* . It uses that (G_c, G_c) is normal. Membership $\tilde{w}_{-1} \in (G_c, G_c)$ is not needed for this step.
5. *Unused lemmas*. [Lemmas 3.1, 3.10, 3.19 and 3.21](#) are proved but not used in the final chain. Instead, [Lemmas 3.20 and 3.22](#) apply (Re:3) and (Im:3) directly with the substituted parameter.

The hypothesis $c(2) \geq 1$ is used only to make $(0, 2, 1)$ and $(1, 2, 1)$ valid.

Lemma 3.1 (mk_rel_one). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ and $r \in F_c$. If $r \in \mathcal{R}_c$, then $\pi(r) = 1$ in G_c .*

Lean: mk_rel_one — proved by AProver, job 1, session 1

```

theorem mk_rel_one (c : ℕ → ℕ) (r : GM.F c) (h : GM.Rel c r) :
  PresentedGroup.mk (GM.Rels c) r = 1 := by
  apply PresentedGroup.one_of_mem
  exact h

```

Proof. Every relator lies in the normal closure $N_{\mathcal{R}_c}$, which is the kernel of π . □

Lemma 3.2 (mk_eqr). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ and $a, b \in F_c$. If $\rho(a, b) = ab^{-1} \in \mathcal{R}_c$, then $\pi(a) = \pi(b)$.*

Lean: mk_eqr — proved by AIProver, job 1, session 1

```

theorem mk_eqr (c : ℕ → ℕ) (a b : GM.F c) (h : GM.Rel c (GM.eqr a b)) :
  PresentedGroup.mk (GM.Rels c) a = PresentedGroup.mk (GM.Rels c) b := by
  exact (PresentedGroup.mk_eq_mk_of_mul_inv_mem h)

```

Proof. We have $ab^{-1} \in \mathcal{R}_c \subseteq \ker \pi$. Hence $\pi(a)\pi(b)^{-1} = 1$, i.e. $\pi(a) = \pi(b)$. $\square$

Lemma 3.3 (subgroup_top_of_gens). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ and let N be a subgroup of G_c . If $\pi(x) \in N$ for every $x \in \mathcal{X}_c$, then $N = G_c$.*

Lean: subgroup_top_of_gens — proved by AIProver, job 1, session 1

```

theorem subgroup_top_of_gens (c : ℕ → ℕ) (N : Subgroup (GM.Grp c))
  (h : ∀ g : GM.Gen c, (PresentedGroup.of g : GM.Grp c) ∈ N) : N = ⊤ := by
  have hgen : Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c → GM.Grp c) ⊆
  ↪ N.carrier := by
    rintro _ ⟨g, rfl⟩
    exact h g
  have hclosure : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) :
  ↪ GM.Gen c → GM.Grp c)) ≤ N :=
    ((Subgroup.closure_le (K := N)).mpr hgen)
  have htop : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c
  ↪ → GM.Grp c)) = ⊤ :=
    PresentedGroup.closure_range_of (GM.Rels c)
  rw [htop] at hclosure
  exact top_le_iff.mp hclosure

```

Proof. The subgroup generated by $\pi(\mathcal{X}_c)$ is all of G_c , since π is surjective and F_c is generated by $\mathcal{X}_c$. This generated subgroup is contained in N by hypothesis. $\square$

Lemma 3.4 (cm_mem_commutator). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ and $a, b \in F_c$. Then $\pi((a, b)) \in (G_c, G_c)$.*

Lean: cm_mem_commutator — proved by AIProver, job 1, session 0

```

theorem cm_mem_commutator (c : ℕ → ℕ) (a b : GM.F c) :
  PresentedGroup.mk (GM.Rels c) (GM.cm a b) ∈ commutator (GM.Grp c) := by
  have h : PresentedGroup.mk (GM.Rels c) (GM.cm a b) = [[PresentedGroup.mk (GM.Rels c) a,
  PresentedGroup.mk (GM.Rels c) b]] := by
    simp [GM.cm, commutatorElement_def]
  rw [h]
  exact Subgroup.commutator_mem_commutator (Subgroup.mem_top _) (Subgroup.mem_top _)

```

Proof. Since π is a homomorphism, $\pi((a, b)) = (\pi(a), \pi(b))$, which is a commutator in G_c . $\square$

Lemma 3.5 (xm_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ with $c(2) \geq 1$. Then for every $u \in \mathbb{C}$, $\pi(X_{-1}(u)) \in (G_c, G_c)$.*

Lean: xm_mem — proved by AIProver, job 1, session 1

```

theorem xm_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xm u) ∈ commutator (GM.Grp c) := by
  set G := GM.Grp c
  let m := PresentedGroup.mk (GM.Rels c)
  have hvalid1 : GM.Valid c 1 2 1 := by
    dsimp [GM.Valid]
    exact ⟨by decide, by decide, hc⟩
  have hvalid0 : GM.Valid c 0 2 1 := by
    dsimp [GM.Valid]
    exact ⟨by decide, by decide, hc⟩
  have hrel : GM.Rel c (GM.eqr (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) (GM.xm ((u :
    ↪ ℂ) * 1))) :=
    GM.Rel.U3a 1 hvalid1 hvalid0 (u : ℂ) 1
  have h_eq : m (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) = m (GM.xm ((u : ℂ) * 1)) := by
    apply mk_eqr c _ _ hrel
  have h_cm_mem : m (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) ∈ commutator G := by
    rw [commutator_eq_closure G]
    apply Subgroup.subset_closure
    exact commutator_mem_commutatorSet (m (GM.xx 1 2 1 hvalid1 (u : ℂ))) (m (GM.yy 0 2 1 hvalid0 1))
  have h_target : m (GM.xm u) = m (GM.xm ((u : ℂ) * 1)) := by simp
  rw [h_target]
  rw [← h_eq]
  exact h_cm_mem

```

Proof. Since $c(2) \geq 1$, the triples $(1, 2, 1)$ and $(0, 2, 1)$ are valid. Relation (U:3a) with $k = 1$ and $v = 1$ gives

$$\pi((X_{1,21}(u), Y_{0,21}(1))) = \pi(X_{-1}(u \cdot 1)) = \pi(X_{-1}(u))$$

by Lemma 3.2. The left-hand side is the commutator of $\pi(X_{1,21}(u))$ and $\pi(Y_{0,21}(1))$, so it lies in (G_c, G_c) . $\square$

Lemma 3.6 (ym_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ with $c(2) \geq 1$. Then for every $u \in \mathbb{C}$, $\pi(Y_{-1}(u)) \in (G_c, G_c)$.*

Lean: ym_mem — proved by AIProver, job 1, session 1

```

theorem ym_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.ym u) ∈ commutator (GM.Grp c) := by
  have hvalid0 : GM.Valid c 0 2 1 := by
    refine ⟨by decide, by decide, ?_⟩
    exact Nat.le_trans (by decide) hc
  have hvalid1 : GM.Valid c 1 2 1 := by
    refine ⟨by decide, by decide, ?_⟩
    exact Nat.le_trans (by decide) hc
  have h_rel : GM.Rel c (GM.eqr (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1))
    (GM.ym (u * 1))) :=
    GM.Rel.U3b 1 hvalid0 hvalid1 u 1
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1)) =
    PresentedGroup.mk (GM.Rels c) (GM.ym (u * 1)) :=
    mk_eqr c _ _ h_rel
  have h_cm_mem : PresentedGroup.mk (GM.Rels c)
    (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1)) ∈ commutator (GM.Grp c) :=
    cm_mem_commutator c _ _
  rw [h_eq] at h_cm_mem
  simp [mul_one] using h_cm_mem

```

Proof. As in [Lemma 3.5](#), $(0, 2, 1)$ and $(1, 2, 1)$ are valid. Relation (U:3b) with $k = 1$ and $v = 1$, together with [Lemma 3.2](#), gives

$$\pi((X_{0,21}(u), Y_{1,21}(1))) = \pi(Y_{-1}(u)).$$

The left-hand side lies in (G_c, G_c) by [Lemma 3.4](#). $\square$

Lemma 3.7 (wm_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ with $c(2) \geq 1$. Then for every $s \in \mathbb{C}^\times$, $\pi(\tilde{w}_{-1}(s)) \in (G_c, G_c)$.*

Lean: wm_mem — proved by AIProver, job 1, session 1

```

theorem wm_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.wm s) ∈ commutator (GM.Grp c) := by
have hx : PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) ∈ commutator (GM.Grp c) :=
  xm_mem c hc (s : ℂ)
have hy : PresentedGroup.mk (GM.Rels c) (GM.ym (-(s : ℂ)-1)) ∈ commutator (GM.Grp c) :=
  ym_mem c hc (-(s : ℂ)-1)
have h_eq : PresentedGroup.mk (GM.Rels c) (GM.wm s) =
  PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) *
  PresentedGroup.mk (GM.Rels c) (GM.ym (-(s : ℂ)-1)) *
  PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) := by
  simp [GM.wm, GM.xm, GM.ym, map_mul]
rw [h_eq]
exact Subgroup.mul_mem _ (Subgroup.mul_mem _ hx hy) hx

```

Proof. We have $\pi(\tilde{w}_{-1}(s)) = \pi(X_{-1}(s))\pi(Y_{-1}(-s^{-1}))\pi(X_{-1}(s))$. Each factor lies in (G_c, G_c) by [Lemmas 3.5](#) and [3.6](#), and (G_c, G_c) is a subgroup. $\square$

Lemma 3.8 (exists_unit_pow_ne_one). *For every $d \in \mathbb{N}$ with $d > 0$ there is $s \in \mathbb{C}^\times$ with $s^d \neq 1$.*

Lean: exists_unit_pow_ne_one — proved by AIProver, job 1, session 1

```

theorem exists_unit_pow_ne_one (d : ℕ) (hd : 0 < d) :
  ∃ s : ℂ×, (s : ℂ) ^ d ≠ 1 := by
have h2val : (2 : ℂ) ^ d ≠ 1 := by
  intro h
  have hnormSq : Complex.normSq ((2 : ℂ) ^ d) = Complex.normSq (1 : ℂ) := by rw [h]
  have h_one : Complex.normSq (1 : ℂ) = 1 := by simp
  rw [h_one] at hnormSq
  have hcalc : Complex.normSq ((2 : ℂ) ^ d) = (Complex.normSq (2 : ℂ)) ^ d := by
    simp using map_pow Complex.normSq (2 : ℂ) d
  have h4 : Complex.normSq (2 : ℂ) = (4 : ℝ) := by norm_num
  rw [hcalc, h4] at hnormSq

  have h4pos : (0 : ℝ) < 4 := by norm_num
  have h4dpos : 0 < (4 : ℝ) ^ d := pow_pos h4pos d
  have h4dgt1 : (1 : ℝ) < (4 : ℝ) ^ d := by
    have h_nat : 1 < (4 : ℕ) ^ d := Nat.one_lt_pow (by omega : d ≠ 0) (by norm_num : 1 < 4)
    exact_mod_cast h_nat
  linarith
  refine ⟨Units.mk0 (2 : ℂ) (by norm_num), h2val⟩

```

Proof. Take $s = 2$. If $2^d = 1$, then taking squared absolute values gives $4^d = 1$. But $4^d > 1$ for $d > 0$, a contradiction. $\square$

Lemma 3.9 (xx_add_mk). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid and $u, v \in \mathbb{C}$. Then*

$$\pi(X_{\ell,jk}(u + v)) = \pi(X_{\ell,jk}(u)) \pi(X_{\ell,jk}(v)).$$

Lean: xx_add_mk — proved by AIProver, job 1, session 1

```

theorem xx_add_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (u + v)) =
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *
      PresentedGroup.mk (GM.Rels c) (GM.xx l j k h v) := by
have hrel : GM.Rel c (GM.eqr (GM.xx l j k h u * GM.xx l j k h v) (GM.xx l j k h (u + v))) :=
  GM.Rel.Im1a l j k h u v
have h_eq := mk_eqr c (GM.xx l j k h u * GM.xx l j k h v) (GM.xx l j k h (u + v)) hrel
calc
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (u + v))
    = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u * GM.xx l j k h v) := by
  rw [← h_eq]
_ = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h v) := by
  rw [map_mul]

```

Proof. Apply Lemma 3.2 to the relator (Im:1a), and use that π is multiplicative. □

Lemma 3.10 (yy_add_mk). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid and $u, v \in \mathbb{C}$. Then*

$$\pi(Y_{\ell,jk}(u + v)) = \pi(Y_{\ell,jk}(u)) \pi(Y_{\ell,jk}(v)).$$

Lean: yy_add_mk — proved by AIProver, job 1, session 0

```

theorem yy_add_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (u + v)) =
    PresentedGroup.mk (GM.Rels c) (GM.yy l j k h u) *
      PresentedGroup.mk (GM.Rels c) (GM.yy l j k h v) := by
have hrel : GM.Rel c (GM.eqr (GM.yy l j k h u * GM.yy l j k h v) (GM.yy l j k h (u + v))) :=
  GM.Rel.Im1b l j k h u v
have := mk_eqr c (GM.yy l j k h u * GM.yy l j k h v) (GM.yy l j k h (u + v)) hrel
simp [GM.eqr, mul_inv_rev] using this.symm

```

Proof. Apply Lemma 3.2 to the relator (Im:1b). □

Lemma 3.11 (xx_sub_mk). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid and $a, b \in \mathbb{C}$. Then*

$$\pi(X_{\ell,jk}(a - b)) = \pi(X_{\ell,jk}(a)) \pi(X_{\ell,jk}(b))^{-1}.$$

Lean: xx_sub_mk — proved by AIProver, job 1, session 1

```

theorem xx_sub_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (a b : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (a - b)) =
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h a) *
      (PresentedGroup.mk (GM.Rels c) (GM.xx l j k h b))-1 := by
have hzero : PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (0 : ℂ)) = 1 := by
have hrel : GM.Rel c (GM.eqr (GM.xx l j k h (0 : ℂ)) 1) := by
have := GM.Rel.Im1a l j k h (0 : ℂ) (0 : ℂ)

```

```

    simp [GM.eqr] using this
    have hmem : GM.eqr (GM.xx 1 j k h (0 : ℂ)) 1 ∈ GM.Rels c := hrel
    exact PresentedGroup.one_of_mem hmem
    have hneg (u : ℂ) : PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (-u)) =
      (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u))-1 := by
    have h := xx_add_mk c 1 j k h u (-u)

    have hzero' : (u + (-u : ℂ)) = (0 : ℂ) := by ring
    rw [hzero'] at h
    rw [hzero] at h

    exact eq_inv_of_mul_eq_one_right h.symm
  calc
    PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a - b))
      = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a + (-b))) := by ring
    _ = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
      PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (-b)) := xx_add_mk c 1 j k h a (-b)
    _ = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
      (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h b))-1 := by rw [hneg b]

```

Proof. Write $X = X_{\ell,j,k}$.

1. The relator (Im:1a) with $u = v = 0$ is $X(0)X(0)X(0)^{-1}$, which equals $X(0)$ in F_c . Hence $\pi(X(0)) = 1$.
2. By Lemma 3.9, $\pi(X(u))\pi(X(-u)) = \pi(X(0)) = 1$, so $\pi(X(-u)) = \pi(X(u))^{-1}$.
3. Therefore $\pi(X(a - b)) = \pi(X(a + (-b))) = \pi(X(a))\pi(X(-b)) = \pi(X(a))\pi(X(b))^{-1}$, again by Lemma 3.9.

□

Lemma 3.12 (yy_sub_mk). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid and $a, b \in \mathbb{C}$. Then*

$$\pi(Y_{\ell,j,k}(a - b)) = \pi(Y_{\ell,j,k}(a)) \pi(Y_{\ell,j,k}(b))^{-1}.$$

Lean: yy_sub_mk — proved by AIProver, job 2, session 0

```

theorem yy_sub_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (a b : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) =
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h a) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
  have hRel : GM.Rel c (GM.eqr (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) (GM.yy 1 j k h a)) := by
    have := GM.Rel.Im1b 1 j k h (a - b) b
    simp [sub_add_cancel] using this
  have hEq := mk_eqr c (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) (GM.yy 1 j k h a) hRel

  have hMul : PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) =
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) *
      PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b) := by
    simp
    rw [hMul] at hEq

```

```

calc
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b))
    = (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) *
      PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b)) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
    simp
  _ = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h a) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
    rw [hEq]

```

Proof. Relation (Im:1b) with parameters $a - b$ and b , together with [Lemma 3.2](#), gives

$$\pi(Y_{\ell,jk}(a - b)) \pi(Y_{\ell,jk}(b)) = \pi(Y_{\ell,jk}(a)).$$

Multiply on the right by $\pi(Y_{\ell,jk}(b))^{-1}$. □

Lemma 3.13 (xx_conj_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid, $s \in \mathbb{C}^\times$ and $v \in \mathbb{C}$. Then*

$$\pi(X_{\ell,jk}(s^{j-\ell}v)) \pi(X_{\ell,jk}(v))^{-1} \in (G_c, G_c).$$

Lean: xx_conj_mem — proved by AIProver, job 2, session 1

```

theorem xx_conj_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^ (j - 1) * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
  have him6b := GM.Rel.Im6b 1 j k h s v
  have hmem : GM.eqr (GM.h2 s * GM.xx 1 j k h v * (GM.h2 s)-1) (GM.xx 1 j k h ((s : ℂ) ^ (j - 1) * v)) ∈
  → GM.Rels c := him6b
  have heq := PresentedGroup.mk_eq_mk_of_mul_inv_mem hmem
  rw [← heq]
  simp [map_mul, map_inv, commutatorElement_def] using Subgroup.commutator_mem_commutator
  (g1 := PresentedGroup.mk (GM.Rels c) (GM.h2 s))
  (g2 := PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v))
  (h1 := show PresentedGroup.mk (GM.Rels c) (GM.h2 s) ∈ (T : Subgroup (GM.Grp c)) from Subgroup.mem_top _)
  (h2 := show PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v) ∈ (T : Subgroup (GM.Grp c)) from
  → Subgroup.mem_top _)

```

Proof. By (Im:6b) and [Lemma 3.2](#),

$$\pi(X_{\ell,jk}(s^{j-\ell}v)) = \pi(H_2(s)) \pi(X_{\ell,jk}(v)) \pi(H_2(s))^{-1}.$$

Hence the element in question is the commutator $(\pi(H_2(s)), \pi(X_{\ell,jk}(v)))$, which lies in (G_c, G_c) . □

Lemma 3.14 (yy_conj_mem_aiprover_s0). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid, $s \in \mathbb{C}^\times$ and $v \in \mathbb{C}$. Then*

$$\pi(Y_{\ell,jk}((s^{j-\ell})^{-1}v)) \pi(Y_{\ell,jk}(v))^{-1} \in (G_c, G_c).$$

Lean: yy_conj_mem_aiprover_s0 — helper of yy_conj_mem, proved by AIProver, job 1, session 0

```

open GM in
theorem yy_conj_mem_aiprover_s0 (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (((s : ℂ) ^ (j - 1))-1 * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy l j k h v))-1 ∈ commutator (GM.Grp c) := by
  have h_rel := Rel.Im6d l j k h s v
  have h_eq := mk_eqr c (GM.h2 s * GM.yy l j k h v * (GM.h2 s)-1)
    (GM.yy l j k h (((s : ℂ) ^ (j - 1))-1 * v)) h_rel
  have h_mk : PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (((s : ℂ) ^ (j - 1))-1 * v)) =
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy l j k h v) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 := by
  calc
    PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (((s : ℂ) ^ (j - 1))-1 * v)) =
      PresentedGroup.mk (GM.Rels c) (GM.h2 s * GM.yy l j k h v * (GM.h2 s)-1) := h_eq.symm
    _ = (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy l j k h v) : GM.Grp c) *
      (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 := by simp [map_mul]
  rw [h_mk]
  have h_comm : (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy l j k h v) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 *
    (PresentedGroup.mk (GM.Rels c) (GM.yy l j k h v) : GM.Grp c)-1 =
    [(PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c),
      (PresentedGroup.mk (GM.Rels c) (GM.yy l j k h v) : GM.Grp c)] := rfl
  rw [h_comm]
  dsimp [commutator]
  apply Subgroup.subset_closure
  exact ⟨(PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c), Subgroup.mem_top _,
    (PresentedGroup.mk (GM.Rels c) (GM.yy l j k h v) : GM.Grp c), Subgroup.mem_top _, rfl⟩

```

Proof. By (Im:6d) and [Lemma 3.2](#),

$$\pi(Y_{\ell,jk}((s^{j-\ell})^{-1}v)) = \pi(H_2(s))\pi(Y_{\ell,jk}(v))\pi(H_2(s))^{-1}.$$

So the element is the commutator $(\pi(H_2(s)), \pi(Y_{\ell,jk}(v))) \in (G_c, G_c)$. □

Lemma 3.15 (yy_conj_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid, $s \in \mathbb{C}^\times$ and $v \in \mathbb{C}$. Then*

$$\pi(Y_{\ell,jk}((s^{j-\ell})^{-1}v))\pi(Y_{\ell,jk}(v))^{-1} \in (G_c, G_c).$$

Lean: yy_conj_mem — proved by AIProver, job 1, session 0

```

theorem yy_conj_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h (((s : ℂ) ^ (j - 1))-1 * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy l j k h v))-1 ∈ commutator (GM.Grp c) := by
  first
  | exact yy_conj_mem_aiprover_s0
  | (intros; apply yy_conj_mem_aiprover_s0 <;> assumption)

```

Proof. This is [Lemma 3.14](#). □

Lemma 3.16 (xx_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid and $u \in \mathbb{C}$. Then $\pi(X_{\ell,jk}(u)) \in (G_c, G_c)$.*

Lean: xx_mem — proved by AIProver, job 3, session 0

```

theorem xx_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) ∈ commutator (GM.Grp c) := by
  have hpos_or : j - 1 = 0 ∨ 0 < j - 1 := Nat.eq_zero_or_pos (j - 1)
  rcases hpos_or with (hjl | hdpos)
  · rcases h with ⟨hlt, _, _⟩
    have : 1 < 1 :=
      Nat.lt_of_lt_of_le hlt (Nat.le_of_sub_eq_zero hjl)
    exact absurd this (Nat.lt_irrefl _)
  · rcases exists_unit_pow_ne_one (j - 1) hdpos with ⟨s, hs⟩
    have hden_ne_zero : (s : ℂ) ^ (j - 1) - 1 ≠ 0 := by
      intro hzero
      have h_one : (s : ℂ) ^ (j - 1) = 1 := sub_eq_zero.mp hzero
      exact hs h_one
    set v := u / ((s : ℂ) ^ (j - 1) - 1) with hv_def
    have h_eq : u = ((s : ℂ) ^ (j - 1) - 1) * v := by
      dsimp [v]
      field_simp [hden_ne_zero]
    rw [h_eq]
    have : ((s : ℂ) ^ (j - 1) - 1) * v = ((s : ℂ) ^ (j - 1)) * v - v := by ring
    rw [this]
    rw [xx_sub_mk c l j k h ((s : ℂ) ^ (j - 1)) * v] v
    exact xx_conj_mem c l j k h s v

```

Proof. Validity gives $\ell < j$, so $d := j - \ell > 0$.

1. By [Lemma 3.8](#) choose $s \in \mathbb{C}^\times$ with $s^d \neq 1$.
2. Put $v := u/(s^d - 1)$. Then $u = (s^d - 1)v = s^d v - v$.
3. By [Lemma 3.11](#), $\pi(X_{\ell,jk}(u)) = \pi(X_{\ell,jk}(s^d v)) \pi(X_{\ell,jk}(v))^{-1}$.
4. This lies in (G_c, G_c) by [Lemma 3.13](#).

No hypothesis on $c(2)$ is needed. □

Lemma 3.17 (yy_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid and $u \in \mathbb{C}$. Then $\pi(Y_{\ell,jk}(u)) \in (G_c, G_c)$.*

Lean: yy_mem — proved by the captain after a hand-back

```

theorem yy_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h u) ∈ commutator (GM.Grp c) := by
  have hjl : 0 < j - 1 := by
    have := h.1
    omega
  obtain ⟨s, hs⟩ := exists_unit_pow_ne_one (j - 1) hjl
  have ht : ((s : ℂ) ^ (j - 1))-1 - 1 ≠ 0 := by
    intro h0
    apply hs
    have h1 : ((s : ℂ) ^ (j - 1))-1 = 1 := sub_eq_zero.mp h0
    first
      | exact inv_eq_one.mp h1
      | simp using h1
  have hu : ((s : ℂ) ^ (j - 1))-1 * (u / (((s : ℂ) ^ (j - 1))-1 - 1)) -
    u / (((s : ℂ) ^ (j - 1))-1 - 1) = u := by

```

```

    rw [← sub_one_mul]
    first
    | exact mul_div_cancel₀ u ht
    | (rw [mul_comm]; exact div_mul_cancel₀ u ht)
    | field_simp
  have key := yy_conj_mem c l j k h s (u / (((s : ℂ) ^ (j - 1))⁻¹ - 1))
  first
  | (rw [← yy_sub_mk c l j k h, hu] at key; exact key)
  | (rw [← hu, yy_sub_mk c l j k h]; exact key)

```

Proof. Let $d := j - \ell > 0$.

1. By [Lemma 3.8](#) choose $s \in \mathbb{C}^\times$ with $s^d \neq 1$. Then $t := (s^d)^{-1} \neq 1$.
2. Put $v := u/(t - 1)$, so that $tv - v = u$.
3. By [Lemma 3.12](#), $\pi(Y_{\ell,jk}(u)) = \pi(Y_{\ell,jk}(tv))\pi(Y_{\ell,jk}(v))^{-1}$.
4. This lies in (G_c, G_c) by [Lemma 3.15](#).

□

Lemma 3.18 (ww_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid and $s \in \mathbb{C}^\times$. Then $\pi(\tilde{w}_{\ell,jk}(s)) \in (G_c, G_c)$.*

Lean: ww_mem — proved by AIProver, job 1, session 0

```

theorem ww_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.ww l j k h s) ∈ commutator (GM.Grp c) := by
  let f : GM.F c →* GM.Grp c := PresentedGroup.mk (GM.Rels c)
  have hxx : f (GM.xx l j k h (s : ℂ)) ∈ commutator (GM.Grp c) := by
    dsimp [f]
    simpa using xx_mem c l j k h (s : ℂ)
  have hyy : f (GM.yy l j k h (-(s : ℂ)-1 / GM.cc l j)) ∈ commutator (GM.Grp c) := by
    simpa using yy_mem c l j k h (-(s : ℂ)-1 / GM.cc l j)

  have hprod : f (GM.xx l j k h (s : ℂ) * GM.yy l j k h (-(s : ℂ)-1 / GM.cc l j) * GM.xx l j k h (s : ℂ))
  → ∈ commutator (GM.Grp c) := by
    have hcalc : f (GM.xx l j k h (s : ℂ) * GM.yy l j k h (-(s : ℂ)-1 / GM.cc l j) * GM.xx l j k h (s :
  → ℂ)) =
      f (GM.xx l j k h (s : ℂ)) * f (GM.yy l j k h (-(s : ℂ)-1 / GM.cc l j)) * f (GM.xx l j k h (s :
  → ℂ)) := by
      simp [map_mul]
      rw [hcalc]
      exact Subgroup.mul_mem _ (Subgroup.mul_mem _ hxx hyy) hxx

  have hww : GM.ww l j k h s = GM.xx l j k h (s : ℂ) * GM.yy l j k h (-(s : ℂ)-1 / GM.cc l j) * GM.xx l j
  → k h (s : ℂ) := rfl
  rw [hww]
  exact hprod

```

Proof. We have

$$\pi(\tilde{w}_{\ell,jk}(s)) = \pi(X_{\ell,jk}(s))\pi(Y_{\ell,jk}(-s^{-1}/c_{\ell j}))\pi(X_{\ell,jk}(s)).$$

Each factor lies in (G_c, G_c) by [Lemmas 3.16](#) and [3.17](#).

□

Lemma 3.19 (h1h2_neg_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ with $c(2) \geq 1$, and $s \in \mathbb{C}^\times$. Then $\pi(H_1(-s)H_2(-s^{-1})) \in (G_c, G_c)$.*

Lean: h1h2_neg_mem — proved by AIProver, job 1, session 1

```

theorem h1h2_neg_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (-s) * GM.h2 (-(s-1))) ∈ commutator (GM.Grp c) := by
  have hrel : GM.Rel c (GM.eqr (GM.wm s * GM.wm 1) (GM.h1 (-s) * GM.h2 (-(s-1)))) :=
    GM.Rel.Re3 s
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.wm s * GM.wm 1) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 (-s) * GM.h2 (-(s-1))) :=
    mk_eqr c (GM.wm s * GM.wm 1) (GM.h1 (-s) * GM.h2 (-(s-1))) hrel
  have h_wm_s : PresentedGroup.mk (GM.Rels c) (GM.wm s) ∈ commutator (GM.Grp c) :=
    wm_mem c hc s
  have h_wm_1 : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) :=
    wm_mem c hc 1
  have h_prod : PresentedGroup.mk (GM.Rels c) (GM.wm s * GM.wm 1) ∈ commutator (GM.Grp c) :=
    Subgroup.mul_mem _ h_wm_s h_wm_1
  rw [← h_eq]
  exact h_prod

```

Proof. By (Re:3) and [Lemma 3.2](#), $\pi(H_1(-s)H_2(-s^{-1})) = \pi(\tilde{w}_{-1}(s))\pi(\tilde{w}_{-1}(1))$. Both factors lie in (G_c, G_c) by [Lemma 3.7](#). $\square$

Lemma 3.20 (h1h2_inv_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ with $c(2) \geq 1$, and $x \in \mathbb{C}^\times$. Then $\pi(H_1(x)H_2(x^{-1})) \in (G_c, G_c)$.*

Lean: h1h2_inv_mem — proved by AIProver, job 2, session 0

```

theorem h1h2_inv_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (x : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 x * GM.h2 x-1) ∈ commutator (GM.Grp c) := by
  have hrel := GM.Rel.Re3 (c := c) (-x)
  have h_simplified : (GM.h1 (c := c) (-(-x)) * GM.h2 (c := c) (-((-x)-1))) = (GM.h1 (c := c) x * GM.h2 (c := c) x-1) := by
    simp
  have hmem : PresentedGroup.mk (GM.Rels c) (GM.wm (-x) * GM.wm 1) ∈ commutator (GM.Grp c) := by
    have h1 : PresentedGroup.mk (GM.Rels c) (GM.wm (-x)) ∈ commutator (GM.Grp c) := wm_mem c hc (-x)
    have h2 : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) := wm_mem c hc 1
    exact Subgroup.mul_mem _ h1 h2
  have heq := mk_eqr c (GM.wm (-x) * GM.wm 1) (GM.h1 x * GM.h2 x-1) (by
    simpa [h_simplified] using hrel)
  rw [← heq]
  exact hmem

```

Proof. Relation (Re:3) with $s = -x$ reads

$$\tilde{w}_{-1}(-x) \tilde{w}_{-1} = H_1(x)H_2(x^{-1}),$$

since $-(-x) = x$ and $-((-x)^{-1}) = x^{-1}$. By [Lemma 3.2](#), $\pi(H_1(x)H_2(x^{-1})) = \pi(\tilde{w}_{-1}(-x))\pi(\tilde{w}_{-1}(1))$. This lies in (G_c, G_c) by [Lemma 3.7](#). $\square$

Lemma 3.21 (im3_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, (ℓ, j, k) valid and $s \in \mathbb{C}^\times$. Then*

$$\pi(H_1((-s)^{j-\ell})H_2((-s)^{\ell+1})) \in (G_c, G_c).$$

Lean: im3_mem — proved by AProver, job 1, session 1

```

theorem im3_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))
    ∈ commutator (GM.Grp c) := by
  set w1 := GM.ww l j k h (s ^ ((1 + 1) * (j - 1))) with hw1
  set w0 := GM.ww l j k h 1 with hw0
  have hIm3_rel : GM.Rel c (GM.eqr (w1 * w0) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))) :=
    GM.Rel.Im3 l j k h s
  have h_mul_inv : w1 * w0 * (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))-1 ∈ GM.Rels c := by
    simp [GM.Rels] using hIm3_rel
  have h_mem' : PresentedGroup.mk (GM.Rels c) (GM.eqr (w1 * w0) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 +
    → 1)))) = 1 :=
    PresentedGroup.one_of_mem hIm3_rel
  have h_eq : PresentedGroup.mk (GM.Rels c) (w1 * w0) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1))) := by
    apply PresentedGroup.mk_eq_mk_of_mul_inv_mem h_mul_inv
  have h_prod_mem : PresentedGroup.mk (GM.Rels c) (w1 * w0) ∈ commutator (GM.Grp c) := by
    have h1 : PresentedGroup.mk (GM.Rels c) w1 ∈ commutator (GM.Grp c) :=
      ww_mem c l j k h (s ^ ((1 + 1) * (j - 1)))
    have h2 : PresentedGroup.mk (GM.Rels c) w0 ∈ commutator (GM.Grp c) :=
      ww_mem c l j k h 1
    exact Subgroup.mul_mem (commutator (GM.Grp c)) h1 h2
  rw [← h_eq]
  exact h_prod_mem

```

Proof. By (Im:3) and [Lemma 3.2](#), the element equals $\pi(\tilde{w}_{\ell,jk}(s^{(\ell+1)(j-\ell)}))\pi(\tilde{w}_{\ell,jk}(1))$. Both factors lie in (G_c, G_c) by [Lemma 3.18](#). $\square$

Lemma 3.22 (h1sq_h2_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ with $c(2) \geq 1$, and $y \in \mathbb{C}^\times$. Then $\pi(H_1(y^2)H_2(y)) \in (G_c, G_c)$.*

Lean: h1sq_h2_mem — proved by AProver, job 2, session 0

```

theorem h1sq_h2_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (y : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (y ^ 2) * GM.h2 y) ∈ commutator (GM.Grp c) := by
  have hvalid : GM.Valid c 0 2 1 := (by decide, by decide, hc)
  have hw1 := ww_mem c 0 2 1 hvalid (y ^ 2)
  have hw2 := ww_mem c 0 2 1 hvalid 1
  have hprod : PresentedGroup.mk (GM.Rels c)
    (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1) ∈ commutator (GM.Grp c) :=
    (commutator (GM.Grp c)).mul_mem hw1 hw2
  have him3 : GM.Rel c (GM.eqr (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1)
    (GM.h1 (y ^ 2) * GM.h2 y)) := by
    have := GM.Rel.Im3 0 2 1 hvalid (-y)
    simp [show ((-y : ℂ×) : ℂ) ^ ((0+1)*(2-0)) = (y : ℂ)2 by norm_num] using this
  have heq : PresentedGroup.mk (GM.Rels c)
    (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 (y ^ 2) * GM.h2 y) := by
    simp [GM.Rels] using PresentedGroup.mk_eq_mk_of_mul_inv_mem him3
  rw [← heq]
  exact hprod

```

Proof. Since $c(2) \geq 1$, the triple $(0, 2, 1)$ is valid. Relation (Im:3) for $(\ell, j, k) = (0, 2, 1)$ and parameter $s = -y$ reads

$$\tilde{w}_{0,21}(y^2) \tilde{w}_{0,21} = H_1(y^2)H_2(y),$$

because $(-y)^{(0+1)(2-0)} = y^2$, $(-(-y))^2 = y^2$ and $(-(-y))^1 = y$. By [Lemma 3.2](#), $\pi(H_1(y^2)H_2(y)) = \pi(\tilde{w}_{0,21}(y^2))\pi(\tilde{w}_{0,21}(1))$. This lies in (G_c, G_c) by [Lemma 3.18](#). $\square$

Lemma 3.23 (`h1_cube_eq`). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ and $z \in \mathbb{C}^\times$. Then in G_c ,*

$$\pi(H_1(z^3)) = \pi(H_1(z)H_2(z^{-1})) \cdot \pi(H_1((z^{-1})^2)H_2(z^{-1}))^{-1}.$$

Lean: `h1_cube_eq` — proved by AIProver, job 5, session 7

```

theorem h1_cube_eq (c : ℕ → ℕ) (z : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (z ^ 3)) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 z-1) *
      (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 z-1))-1 := by
  let φ := PresentedGroup.mk (GM.Rels c)
  have hφ_mul : ∀ x y : GM.F c, φ (x * y) = φ x * φ y := by
    intro x y; exact MonoidHom.map_mul φ x y
  have hφ_inv : ∀ x : GM.F c, φ (x-1) = (φ x)-1 := by
    intro x; exact MonoidHom.map_inv φ x

  have h1_mul : ∀ s t : ℂ×, φ (GM.h1 (c := c) (s * t)) = φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) t) := by
    intro s t
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) t) (GM.h1 (c := c) (s * t))) :=
    GM.Rel.H1a (c := c) s t
  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)

  have hcalc : φ (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) t) (GM.h1 (c := c) (s * t))) =
    φ (GM.h1 (c := c) s * GM.h1 (c := c) t) * (φ (GM.h1 (c := c) (s * t)))-1 := by
    simp [GM.eqr, hφ_mul, hφ_inv]
  rw [hcalc] at h_one

  have h_eq : φ (GM.h1 (c := c) s * GM.h1 (c := c) t) = φ (GM.h1 (c := c) (s * t)) := by
    calc
      φ (GM.h1 (c := c) s * GM.h1 (c := c) t) =
        (φ (GM.h1 (c := c) s * GM.h1 (c := c) t) * (φ (GM.h1 (c := c) (s * t)))-1) *
          φ (GM.h1 (c := c) (s * t)) := by group
      _ = 1 * φ (GM.h1 (c := c) (s * t)) := by rw [h_one]
      _ = φ (GM.h1 (c := c) (s * t)) := by simp
  rw [hφ_mul] at h_eq
  exact h_eq.symm

  have h1_one : φ (GM.h1 (c := c) 1) = 1 := by
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) (GM.h1 (c := c) (1 * 1))) :=
    GM.Rel.H1a (c := c) 1 1
  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)

```

```

have hcalc :  $\varphi$  (GM.eqr (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) (GM.h1 (c := c) (1 * 1))) =
   $\varphi$  (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) *  $\varphi$  (GM.h1 (c := c) (1 * 1))-1 := by
  simp [GM.eqr, h $\varphi$ _mul]
rw [hcalc] at h_one

rw [mul_one] at h_one

rw [h $\varphi$ _mul] at h_one

calc
   $\varphi$  (GM.h1 (c := c) 1) =  $\varphi$  (GM.h1 (c := c) 1) *  $\varphi$  (GM.h1 (c := c) 1) *  $\varphi$  (GM.h1 (c := c) 1)-1 := by
  ↪ group
  _ = 1 := h_one

have h1_inv :  $\forall s : \mathbb{C}^\times$ ,  $\varphi$  (GM.h1 (c := c) s-1) =  $\varphi$  (GM.h1 (c := c) s)-1 := by
  intro s
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) s-1) (GM.h1 (c := c) (s * s-1))) :=
    GM.Rel.H1a (c := c) s s-1

  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)
  have hcalc :  $\varphi$  (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) s-1) (GM.h1 (c := c) (s * s-1))) =
     $\varphi$  (GM.h1 (c := c) s * GM.h1 (c := c) s-1) *  $\varphi$  (GM.h1 (c := c) (s * s-1))-1 := by
    simp [GM.eqr, h $\varphi$ _mul, h $\varphi$ _inv]
  rw [hcalc] at h_one

have h_s_one :  $\varphi$  (GM.h1 (c := c) (s * s-1)) = 1 := by
  have : s * s-1 = (1 :  $\mathbb{C}^\times$ ) := by simp
  rw [this]
  exact h1_one
  rw [h_s_one] at h_one

have : (1 : PresentedGroup (GM.Rels c))-1 = 1 := by simp
  rw [this, mul_one] at h_one

calc
   $\varphi$  (GM.h1 (c := c) s-1) = 1 *  $\varphi$  (GM.h1 (c := c) s-1) := by simp
  _ = (( $\varphi$  (GM.h1 (c := c) s))-1 *  $\varphi$  (GM.h1 (c := c) s)) *  $\varphi$  (GM.h1 (c := c) s-1) := by simp
  _ = ( $\varphi$  (GM.h1 (c := c) s))-1 * ( $\varphi$  (GM.h1 (c := c) s) *  $\varphi$  (GM.h1 (c := c) s-1)) := by group
  _ = ( $\varphi$  (GM.h1 (c := c) s))-1 * 1 := by
    rw [show  $\varphi$  (GM.h1 (c := c) s) *  $\varphi$  (GM.h1 (c := c) s-1) = 1 from h_one]
  _ = ( $\varphi$  (GM.h1 (c := c) s))-1 := by simp

have h_pow_h1 :  $\forall (x : \mathbb{C}^\times) (n : \mathbb{N})$ ,  $\varphi$  (GM.h1 (c := c) (x ^ n)) = ( $\varphi$  (GM.h1 (c := c) x)) ^ n := by
  intro x n
  induction' n with k ih

```

```

    .
    simp [h1_one]
    .
    rw [pow_succ, pow_succ]

    rw [h1_mul (x ^ k) x]
    rw [ih]

have h_inv_h1 : ∀ (x : ℂ×), φ (GM.h1 (c := c) x-1) = (φ (GM.h1 (c := c) x))-1 := h1_inv

calc
  φ (GM.h1 (c := c) (z ^ 3)) = (φ (GM.h1 (c := c) z)) ^ 3 := by rw [h_pow_h1]
  _ = (φ (GM.h1 (c := c) z)) * ((φ (GM.h1 (c := c) z))-1 ^ 2)-1 := by group
  _ = (φ (GM.h1 (c := c) z) * φ (GM.h2 (c := c) z-1)) * ((φ (GM.h1 (c := c) z))-1 ^ 2) * φ (GM.h2 (c :=
↪ c) z-1)-1 := by group
  _ = (φ (GM.h1 (c := c) z) * φ (GM.h2 (c := c) z-1)) * ((φ (GM.h1 (c := c) (z-1 ^ 2)) * φ (GM.h2 (c :=
↪ c) z-1))-1 := by
    simp [h_pow_h1, h_inv_h1]
  _ = PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 z-1) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1 ^ 2) * GM.h2 z-1))-1 := by
    simp [φ, hφ_mul]

```

Proof. Write $\eta(s) := \pi(H_1(s))$.

1. By (H:1a) and [Lemma 3.2](#), $\eta(s)\eta(t) = \eta(st)$ for all $s, t \in \mathbb{C}^\times$.
2. With $s = t = 1$ this gives $\eta(1)^2 = \eta(1)$, hence $\eta(1) = 1$.
3. With $t = s^{-1}$ it gives $\eta(s)\eta(s^{-1}) = 1$, hence $\eta(s^{-1}) = \eta(s)^{-1}$.
4. By induction on n , $\eta(x^n) = \eta(x)^n$ for all $n \in \mathbb{N}$.

Put $a := \eta(z)$ and $b := \pi(H_2(z^{-1}))$. Then $\eta((z^{-1})^2) = (a^{-1})^2$, and the right-hand side equals

$$ab(a^{-2}b)^{-1} = abb^{-1}a^2 = a^3 = \eta(z^3).$$

□

Lemma 3.24 (exists_unit_cube). *For every $s \in \mathbb{C}^\times$ there is $z \in \mathbb{C}^\times$ with $z^3 = s$.*

Lean: exists_unit_cube — proved by AIProver, job 1, session 1

```

theorem exists_unit_cube (s : ℂ×) : ∃ z : ℂ×, z ^ 3 = s := by
  obtain ⟨w, hw⟩ := IsAlgClosed.exists_pow_nat_eq (s : ℂ) (by decide : 0 < 3)
  have hw_ne_zero : w ≠ 0 := by
    intro hzero
    rw [hzero, zero_pow (by decide : 3 ≠ 0)] at hw
  have hs_ne_zero : (s : ℂ) ≠ 0 := Units.ne_zero s
  exact hs_ne_zero (Eq.symm hw)
  let z : ℂ× := Units.mk0 w hw_ne_zero
  refine ⟨z, ?_⟩
  apply Units.ext_iff.mpr
  calc
    (z : ℂ) ^ 3 = w ^ 3 := rfl
    _ = (s : ℂ) := hw

```

Proof. Since $\mathbb{C}$ is algebraically closed, there is $w \in \mathbb{C}$ with $w^3 = s$. Since $s \neq 0$, also $w \neq 0$, so $w \in \mathbb{C}^\times$. $\square$

Lemma 3.25 (h1_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ with $c(2) \geq 1$, and $s \in \mathbb{C}^\times$. Then $\pi(H_1(s)) \in (G_c, G_c)$.*

Lean: h1_mem — proved by AProver, job 3, session 1

```

theorem h1_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s) ∈ commutator (GM.Grp c) := by
  rcases exists_unit_cube s with ⟨z, hz⟩

  have h_comm_z : PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 (z-1)) ∈ commutator (GM.Grp c) :=
    h1h2_inv_mem c hc z

  have h_comm_z_inv : PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 (z-1)) ∈
    commutator (GM.Grp c) :=
    h1sq_h2_mem c hc (z-1)

  have h_cube := h1_cube_eq c z

  have h_prod : PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 (z-1)) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 (z-1)))-1 ∈
    commutator (GM.Grp c) := by
    apply Subgroup.mul_mem
    · exact h_comm_z
    · apply Subgroup.inv_mem
      exact h_comm_z_inv

  have h_cube_mem : PresentedGroup.mk (GM.Rels c) (GM.h1 (z ^ 3)) ∈ commutator (GM.Grp c) := by
    rw [h_cube]
    exact h_prod

  simp [hz] using h_cube_mem

```

Proof. By [Lemma 3.24](#) choose $z \in \mathbb{C}^\times$ with $z^3 = s$. Then:

- $\pi(H_1(z)H_2(z^{-1})) \in (G_c, G_c)$ by [Lemma 3.20](#);
- $\pi(H_1((z^{-1})^2)H_2(z^{-1})) \in (G_c, G_c)$ by [Lemma 3.22](#) with $y = z^{-1}$.

Since (G_c, G_c) is a subgroup, the first element times the inverse of the second lies in (G_c, G_c) . By [Lemma 3.23](#), this product equals $\pi(H_1(z^3)) = \pi(H_1(s))$. $\square$

Lemma 3.26 (h2_eq_conj). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ and $s \in \mathbb{C}^\times$. Then*

$$\pi(H_2(s)) = \pi(\tilde{w}_{-1}) \pi(H_1(s)) \pi(\tilde{w}_{-1})^{-1}.$$

Lean: h2_eq_conj — proved by AProver, job 1, session 1

```

theorem h2_eq_conj (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) =
    PresentedGroup.mk (GM.Rels c) (GM.wm 1) * PresentedGroup.mk (GM.Rels c) (GM.h1 s) *
      (PresentedGroup.mk (GM.Rels c) (GM.wm 1))-1 := by

have h_eq := mk_eqr c (GM.wm 1 * GM.h1 s * (GM.wm 1)-1) (GM.h2 s) (by
  apply GM.Rel.Re5a)

simp [PresentedGroup.mk, map_mul, map_inv] using h_eq.symm

```

Proof. Apply [Lemma 3.2](#) to (Re:5a), and use that π is a homomorphism. $\square$

Lemma 3.27 (h2_mem). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$ with $c(2) \geq 1$, and $s \in \mathbb{C}^\times$. Then $\pi(H_2(s)) \in (G_c, G_c)$.*

Lean: h2_mem — proved by AIProver, job 4, session 1

```

theorem h2_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) ∈ commutator (GM.Grp c) := by
have hnormal : (commutator (GM.Grp c)).Normal := inferInstance
have hh1 : PresentedGroup.mk (GM.Rels c) (GM.h1 s) ∈ commutator (GM.Grp c) := h1_mem c hc s
have hww : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) := wm_mem c hc 1
rw [h2_eq_conj c s]
exact hnormal.conj_mem (PresentedGroup.mk (GM.Rels c) (GM.h1 s)) hh1
  (PresentedGroup.mk (GM.Rels c) (GM.wm 1))

```

Proof. By [Lemma 3.26](#), $\pi(H_2(s))$ is a conjugate of $\pi(H_1(s))$, which lies in (G_c, G_c) by [Lemma 3.25](#). The commutator subgroup is normal, so the conjugate also lies in (G_c, G_c) . $\square$

3.2 Proof of the theorem

Lean: solution — main proof from the captain's sketch

```

theorem solution (c : ℕ → ℕ) (hc : 1 ≤ c 2) :
  commutator (GM.Grp c) = ⊤ := by
  apply subgroup_top_of_gens
  intro g
  cases g with
  | H1 s => exact h1_mem c hc s
  | H2 s => exact h2_mem c hc s
  | Xm u => exact xm_mem c hc u
  | Ym u => exact ym_mem c hc u
  | X l j k h u => exact xx_mem c l j k h u
  | Y l j k h u => exact yy_mem c l j k h u

```

Theorem 3.28. *For every $c : \mathbb{N} \rightarrow \mathbb{N}$ with $c(2) \geq 1$, $(G_c, G_c) = G_c$.*

Proof. By [Lemma 3.3](#) it suffices to show that $\pi(x) \in (G_c, G_c)$ for every generator $x \in \mathcal{X}_c$. We check each kind of generator:

- $x = H_1(s)$: [Lemma 3.25](#);
- $x = H_2(s)$: [Lemma 3.27](#);

- $x = X_{-1}(u)$: [Lemma 3.5](#);
- $x = Y_{-1}(u)$: [Lemma 3.6](#);
- $x = X_{\ell,jk}(u)$ with (ℓ, j, k) valid: [Lemma 3.16](#);
- $x = Y_{\ell,jk}(u)$ with (ℓ, j, k) valid: [Lemma 3.17](#).

For c the sequence of coefficients of J , we have $c(2) = 21493760 \geq 1$, which gives the source's Theorem 3.3. This value is taken from the source and is not checked formally. $\square$

4 Independent review

A referee model (claude-opus-5-5), which took no part in the formalization or the proof, compared the verified Lean file with the source. Verdict: **FAITHFUL**.

Definitions. $\text{GM.Valid } c \mid j \mid k$ is $l < j \wedge 1 \leq k \wedge k \leq c(j)$. This is exactly membership of (ℓ, j, k) in E , with $c : \mathbb{N} \rightarrow \mathbb{N}$ taken as a parameter, as the formalization note permits. For $j = 0$ the condition $l < 0$ is false, so no spurious indices appear. Because the proof argument is a Prop, proof irrelevance means Gen.X and Gen.Y create no duplicate generators.

GM.Gen gives exactly the symbols of $\mathcal{X}$:

- $H_1(s)$ and $H_2(s)$ for $s \in \mathbb{C}^\times$;
- $X_{-1}(u)$ and $Y_{-1}(u)$ for $u \in \mathbb{C}$;
- $X_{\ell,jk}(u)$ and $Y_{\ell,jk}(u)$ for $(\ell, j, k) \in E$.

GM.F is the free group on these symbols. GM.Grp is PresentedGroup of GM.Rels , which is $F(\mathcal{X})/N_{\mathcal{R}}$.

GM.cc is $(-1)^{\ell+1} \binom{j-1}{\ell} (\ell+1)(j-\ell)$. Truncated subtraction is harmless because $\ell < j$. On E the value is nonzero, so the divisions by $c_{\ell j}$ are genuine.

GM.wm and GM.wv match (Re:0) and (Im:0). GM.cm is $aba^{-1}b^{-1}$, the source convention. GM.eqr encodes $a = b$ as the relator ab^{-1} .

I checked every constructor of GM.Rel against the source:

- (H:1a), (H:1b), (H:2) match.
- (Re:1a) through (Re:6d) match. In (Re:2) both s and t range over units, as in the source.
- (Im:1a) through (Im:6d) match. The arguments $-t^{-1}/c_{\ell j}$, $-c_{\ell j}t^2s$ and $t^{-1}/c_{\ell j}$ are correct. In (Im:3) the exponents are $s^{(\ell+1)(j-\ell)}$, $(-s)^{j-\ell}$ and $(-s)^{\ell+1}$. In (Im:5) and (Im:6) the inverses are written as $(s^n)^{-1}$.
- (U:1a) through (U:1d) use the indices $j-1$ and 0 with validity hypotheses.
- (U:2) has the disjunction $j \neq p$, $k \neq q$, or $|\ell - m| > 1$.
- (U:3a) and (U:3b) match.
- (U:4a) and (U:4b) use index $j-1-\ell$ and sign $(-1)^{j-\ell-1}$. The extra validity hypothesis on $j-1-\ell$ is always satisfiable when $(\ell, j, k) \in E$, so no instance is lost.

I found no discrepancy.

Statement. The target is `gsimple_g_m_perfect`: for every $c : \mathbb{N} \rightarrow \mathbb{N}$ with $1 \leq c(2)$, the commutator subgroup of $\text{GM.Grp } c$ is $\top$. This is the statement $G(\mathbf{m}) = (G(\mathbf{m}), G(\mathbf{m}))$.

The only hypothesis is $c(2) \geq 1$, which the formalization note explicitly allows. The true coefficients of J satisfy it, since $c(2) = 21493760$. To recover the source statement one supplies the actual coefficient sequence $c(j)$ for $j \geq 1$. The values $c(0)$ and $c(-1)$ never enter E .

The statement is not vacuous. The hypothesis is satisfiable, and the conclusion concerns the group presented by the faithfully transcribed relations.

Proof. The final step, `subgroup_top_of_gens`, reduces the theorem to showing that every generator lies in the commutator subgroup. This uses `PresentedGroup.closure_range_of`. The remaining lemmas mirror the source proof:

- `xm_mem` and `ym_mem` use (U:3a) and (U:3b) with $k = 1$ and $v = 1$, so $uv = u$. The case $k = 1$ is valid because of the hypothesis $c(2) \geq 1$.
- `xx_mem` and `yy_mem` use (Im:6b) and (Im:6d) together with (Im:1a) and (Im:1b). The additivity relations are packaged as `xx_sub_mk` and `yy_sub_mk`. The proof picks $s = 2$ and uses $|2^d| \neq 1$ to get $s^{j-\ell} \neq 1$; this is `exists_unit_pow_ne_one`. It then solves for v . This is the same argument as the source.
- `wm_mem` and `ww_mem` correspond to the $\tilde{w}$ step of the source.
- `h1h2_inv_mem` uses (Re:3) with $s = -x$.
- `h1sq_h2_mem` uses (Im:3) with $(\ell, j, k) = (0, 2, 1)$ and $s = -y$.
- `h1_cube_eq` derives the cube identity from (H:1a) alone, as in the source.
- `exists_unit_cube` supplies cube roots via algebraic closure. `h1_mem` then follows.
- `h2_mem` uses (Re:5a) and normality of the commutator subgroup. The source instead uses the product of $\tilde{w}_{-1}$, $H_1(s)$ and $\tilde{w}_{-1}^{-1}$; either argument works.

No lemma assumes anything beyond the defining relations and $c(2) \geq 1$. Each lemma states exactly the step it represents.

Concerns.

- The coefficients of J are replaced by an arbitrary c with $c(2) \geq 1$. This is sanctioned by the problem note, and the theorem then applies to the true c .

A Complete Lean file

The verified file, as checked by Lean.

```
import Mathlib
set_option autoImplicit false

namespace GM

/-- Index set E:  $(\ell, j, k)$  with  $0 \leq \ell < j$  and  $1 \leq k \leq c(j)$ . -/
def Valid (c : ℕ → ℕ) (l j k : ℕ) : Prop := 1 < j ∧ 1 ≤ k ∧ k ≤ c j
```

```

/-- The generating symbols. -/
inductive Gen (c : ℕ → ℕ) : Type
| H1 : ℂ× → Gen c
| H2 : ℂ× → Gen c
| Xm : ℂ → Gen c
| Ym : ℂ → Gen c
| X (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : Gen c
| Y (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : Gen c

abbrev F (c : ℕ → ℕ) := FreeGroup (Gen c)

def h1 {c : ℕ → ℕ} (s : ℂ×) : F c := FreeGroup.of (Gen.H1 s)
def h2 {c : ℕ → ℕ} (s : ℂ×) : F c := FreeGroup.of (Gen.H2 s)
def xm {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Xm u)
def ym {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Ym u)
def xx {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.X l j k h u)
def yy {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=
  FreeGroup.of (Gen.Y l j k h u)

/-- c_{ℓ j} = (-1)^(ℓ+1) * binom(j-1,ℓ) * (ℓ+1) * (j-ℓ). -/
noncomputable def cc (l j : ℕ) : ℂ :=
  (-1 : ℂ) ^ (l + 1) * (((j - 1).choose l : ℕ) : ℂ) * ((l : ℂ) + 1) * ((j - 1 : ℕ) : ℂ)

/-- commutator (a,b) = a b a-1 b-1 -/
def cm {c : ℕ → ℕ} (a b : F c) : F c := a * b * a-1 * b-1

/-- relator for the equation a = b -/
def eqr {c : ℕ → ℕ} (a b : F c) : F c := a * b-1

noncomputable def wm {c : ℕ → ℕ} (s : ℂ×) : F c :=
  xm (s : ℂ) * ym (-(s : ℂ)-1) * xm (s : ℂ)

noncomputable def ww {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) : F c :=
  xx l j k h (s : ℂ) * yy l j k h (-(s : ℂ)-1) / cc l j * xx l j k h (s : ℂ)

/-- The defining relators. -/
inductive Rel (c : ℕ → ℕ) : F c → Prop
| H1a (s t : ℂ×) : Rel c (eqr (h1 s * h1 t) (h1 (s * t)))
| H1b (s t : ℂ×) : Rel c (eqr (h2 s * h2 t) (h2 (s * t)))
| H2 (s t : ℂ×) : Rel c (eqr (h1 s * h2 t) (h2 t * h1 s))
| Re1a (u v : ℂ) : Rel c (eqr (xm u * xm v) (xm (u + v)))
| Re1b (u v : ℂ) : Rel c (eqr (ym u * ym v) (ym (u + v)))
| Re2 (s t : ℂ×) : Rel c (eqr (ym (-(t : ℂ)) * xm (s : ℂ) * ym (t : ℂ))
  (xm (-(t : ℂ)-1) * ym (-(t : ℂ) ^ 2 * (s : ℂ)) * xm ((t : ℂ)-1)))
| Re3 (s : ℂ×) : Rel c (eqr (wm s * wm 1) (h1 (-s) * h2 (-(s-1))))
| Re4a (u : ℂ) : Rel c (eqr (wm 1 * xm u * (wm 1)-1) (ym (-u)))
| Re4b (u : ℂ) : Rel c (eqr (wm 1 * ym u * (wm 1)-1) (xm (-u)))
| Re5a (s : ℂ×) : Rel c (eqr (wm 1 * h1 s * (wm 1)-1) (h2 s))
| Re5b (s : ℂ×) : Rel c (eqr (wm 1 * h2 s * (wm 1)-1) (h1 s))
| Re6a (s : ℂ×) (u : ℂ) : Rel c (eqr (h1 s * xm u * (h1 s)-1) (xm ((s : ℂ) * u)))
| Re6b (s : ℂ×) (u : ℂ) : Rel c (eqr (h2 s * xm u * (h2 s)-1) (xm ((s : ℂ)-1 * u)))
| Re6c (s : ℂ×) (u : ℂ) : Rel c (eqr (h1 s * ym u * (h1 s)-1) (ym ((s : ℂ)-1 * u)))
| Re6d (s : ℂ×) (u : ℂ) : Rel c (eqr (h2 s * ym u * (h2 s)-1) (ym ((s : ℂ) * u)))
| Im1a (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqr (xx l j k h u * xx l j k h v) (xx l j k h (u + v)))
| Im1b (l j k : ℕ) (h : Valid c l j k) (u v : ℂ) :
  Rel c (eqr (yy l j k h u * yy l j k h v) (yy l j k h (u + v)))

```

```

| Im2 (l j k : ℕ) (h : Valid c l j k) (s t : ℂ×) :
  Rel c (eqr (yy l j k h (-(t : ℂ)) * xx l j k h (s : ℂ) * yy l j k h (t : ℂ))
    (xx l j k h (-(t : ℂ)-1 / cc l j) *
      yy l j k h (-(cc l j) * (t : ℂ) ^ 2 * (s : ℂ)) *
      xx l j k h ((t : ℂ)-1 / cc l j)))
| Im3 (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) :
  Rel c (eqr (ww l j k h (s ^ ((1 + 1) * (j - 1))) * ww l j k h 1)
    (h1 ((-s) ^ (j - 1)) * h2 ((-s) ^ (1 + 1))))
| Im4a (l j k : ℕ) (h : Valid c l j k) (u : ℂ) :
  Rel c (eqr (ww l j k h 1 * xx l j k h u * (ww l j k h 1)-1) (yy l j k h (-u / cc l j)))
| Im4b (l j k : ℕ) (h : Valid c l j k) (u : ℂ) :
  Rel c (eqr (ww l j k h 1 * yy l j k h u * (ww l j k h 1)-1) (xx l j k h (-(cc l j) * u)))
| Im5a (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) :
  Rel c (eqr (ww l j k h 1 * h1 (s ^ (j - 1)) * (ww l j k h 1)-1) (h2 ((s ^ (1 + 1))-1)))
| Im5b (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) :
  Rel c (eqr (ww l j k h 1 * h2 (s ^ (1 + 1)) * (ww l j k h 1)-1) (h1 ((s ^ (j - 1))-1)))
| Im6a (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h1 s * xx l j k h u * (h1 s)-1) (xx l j k h ((s : ℂ) ^ (1 + 1) * u)))
| Im6b (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h2 s * xx l j k h u * (h2 s)-1) (xx l j k h ((s : ℂ) ^ (j - 1) * u)))
| Im6c (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h1 s * yy l j k h u * (h1 s)-1) (yy l j k h (((s : ℂ) ^ (1 + 1))-1 * u)))
| Im6d (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h2 s * yy l j k h u * (h2 s)-1) (yy l j k h (((s : ℂ) ^ (j - 1))-1 * u)))
| U1a (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (xm u) (xx (j - 1) j k h v))
| U1b (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (ym u) (yy (j - 1) j k h v))
| U1c (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (ym u) (xx 0 j k h v))
| U1d (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (xm u) (yy 0 j k h v))
| U2 (l j k : ℕ) (h : Valid c l j k) (m p q : ℕ) (h' : Valid c m p q) (u v : ℂ)
  (hne : j ≠ p ∨ k ≠ q ∨ 1 < Int.natAbs ((l : ℤ) - (m : ℤ))) :
  Rel c (cm (xx l j k h u) (yy m p q h' v))
| U3a (k : ℕ) (h : Valid c 1 2 k) (h' : Valid c 0 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 1 2 k h u) (yy 0 2 k h' v)) (xm (u * v)))
| U3b (k : ℕ) (h : Valid c 0 2 k) (h' : Valid c 1 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 0 2 k h u) (yy 1 2 k h' v)) (ym (u * v)))
| U4a (l j k : ℕ) (h : Valid c l j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * xx l j k h u * (wm 1)-1)
    (xx (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))
| U4b (l j k : ℕ) (h : Valid c l j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * yy l j k h u * (wm 1)-1)
    (yy (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))

def Rels (c : ℕ → ℕ) : Set (F c) := {r | Rel c r}

/-- The group G(m) = ⟨X | R⟩. -/
abbrev Grp (c : ℕ → ℕ) : Type := PresentedGroup (Rels c)

end GM

theorem mk_rel_one (c : ℕ → ℕ) (r : GM.F c) (h : GM.Rel c r) :
  PresentedGroup.mk (GM.Rels c) r = 1 := by
  apply PresentedGroup.one_of_mem
  exact h

```

```

theorem mk_eqr (c : ℕ → ℕ) (a b : GM.F c) (h : GM.Rel c (GM.eqr a b)) :
  PresentedGroup.mk (GM.Rels c) a = PresentedGroup.mk (GM.Rels c) b := by
  exact (PresentedGroup.mk_eq_mk_of_mul_inv_mem h)

theorem subgroup_top_of_gens (c : ℕ → ℕ) (N : Subgroup (GM.Grp c))
  (h : ∀ g : GM.Gen c, (PresentedGroup.of g : GM.Grp c) ∈ N) : N =  $\overline{\phantom{x}}$  := by
  have hgen : Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c → GM.Grp c) ⊆
  ↪ N.carrier := by
    rintro _ ⟨g, rfl⟩
    exact h g
  have hclosure : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) :
  ↪ GM.Gen c → GM.Grp c)) ≤ N :=
    ((Subgroup.closure_le (K := N)).mpr hgen)
  have htop : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c
  ↪ → GM.Grp c)) =  $\overline{\phantom{x}}$  :=
    PresentedGroup.closure_range_of (GM.Rels c)
  rw [htop] at hclosure
  exact top_le_iff.mp hclosure

theorem cm_mem_commutator (c : ℕ → ℕ) (a b : GM.F c) :
  PresentedGroup.mk (GM.Rels c) (GM.cm a b) ∈ commutator (GM.Grp c) := by
  have h : PresentedGroup.mk (GM.Rels c) (GM.cm a b) = [[PresentedGroup.mk (GM.Rels c) a,
  PresentedGroup.mk (GM.Rels c) b]] := by
    simp [GM.cm, commutatorElement_def]
  rw [h]
  exact Subgroup.commutator_mem_commutator (Subgroup.mem_top _) (Subgroup.mem_top _)

theorem xm_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xm u) ∈ commutator (GM.Grp c) := by
  set G := GM.Grp c
  let m := PresentedGroup.mk (GM.Rels c)
  have hvalid1 : GM.Valid c 1 2 1 := by
    dsimp [GM.Valid]
    exact ⟨by decide, by decide, hc⟩
  have hvalid0 : GM.Valid c 0 2 1 := by
    dsimp [GM.Valid]
    exact ⟨by decide, by decide, hc⟩
  have hrel : GM.Rel c (GM.eqr (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) (GM.xm ((u :
  ↪ ℂ) * 1))) :=
    GM.Rel.U3a 1 hvalid1 hvalid0 (u : ℂ) 1
  have h_eq : m (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) = m (GM.xm ((u : ℂ) * 1)) := by
    apply mk_eqr c _ _ hrel
  have h_cm_mem : m (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) ∈ commutator G := by
    rw [commutator_eq_closure G]
    apply Subgroup.subset_closure
    exact commutator_mem_commutatorSet (m (GM.xx 1 2 1 hvalid1 (u : ℂ))) (m (GM.yy 0 2 1 hvalid0 1))
  have h_target : m (GM.xm u) = m (GM.xm ((u : ℂ) * 1)) := by simp
  rw [h_target]
  rw [← h_eq]
  exact h_cm_mem

theorem ym_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.ym u) ∈ commutator (GM.Grp c) := by
  have hvalid0 : GM.Valid c 0 2 1 := by
    refine ⟨by decide, by decide, ?_⟩
    exact Nat.le_trans (by decide) hc
  have hvalid1 : GM.Valid c 1 2 1 := by
    refine ⟨by decide, by decide, ?_⟩

```

```

    exact Nat.le_trans (by decide) hc
  have h_rel : GM.Rel c (GM.eqr (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1))
    (GM.ym (u * 1))) :=
    GM.Rel.U3b 1 hvalid0 hvalid1 u 1
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1)) =
    PresentedGroup.mk (GM.Rels c) (GM.ym (u * 1)) :=
    mk_eqr c _ _ h_rel
  have h_cm_mem : PresentedGroup.mk (GM.Rels c)
    (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1)) ∈ commutator (GM.Grp c) :=
    cm_mem_commutator c _ _
  rw [h_eq] at h_cm_mem
  simp [mul_one] using h_cm_mem

theorem wm_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.wm s) ∈ commutator (GM.Grp c) := by
  have hx : PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) ∈ commutator (GM.Grp c) :=
    xm_mem c hc (s : ℂ)
  have hy : PresentedGroup.mk (GM.Rels c) (GM.ym (-(s : ℂ)-1)) ∈ commutator (GM.Grp c) :=
    ym_mem c hc (-(s : ℂ)-1)
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.wm s) =
    PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) *
    PresentedGroup.mk (GM.Rels c) (GM.ym (-(s : ℂ)-1)) *
    PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) := by
    simp [GM.wm, GM.xm, GM.ym, map_mul]
  rw [h_eq]
  exact Subgroup.mul_mem _ (Subgroup.mul_mem _ hx hy) hx

theorem exists_unit_pow_ne_one (d : ℕ) (hd : 0 < d) :
  ∃ s : ℂ×, (s : ℂ) ^ d ≠ 1 := by
  have h2val : (2 : ℂ) ^ d ≠ 1 := by
    intro h
    have hnormSq : Complex.normSq ((2 : ℂ) ^ d) = Complex.normSq (1 : ℂ) := by rw [h]
    have h_one : Complex.normSq (1 : ℂ) = 1 := by simp
    rw [h_one] at hnormSq
    have hcalc : Complex.normSq ((2 : ℂ) ^ d) = (Complex.normSq (2 : ℂ)) ^ d := by
      simp using map_pow Complex.normSq (2 : ℂ) d
    have h4 : Complex.normSq (2 : ℂ) = (4 : ℝ) := by norm_num
    rw [hcalc, h4] at hnormSq

    have h4pos : (0 : ℝ) < 4 := by norm_num
    have h4dpos : 0 < (4 : ℝ) ^ d := pow_pos h4pos d
    have h4dgt1 : (1 : ℝ) < (4 : ℝ) ^ d := by
      have h_nat : 1 < (4 : ℕ) ^ d := Nat.one_lt_pow (by omega : d ≠ 0) (by norm_num : 1 < 4)
      exact_mod_cast h_nat
    linarith
  refine ⟨Units.mk0 (2 : ℂ) (by norm_num), h2val⟩

theorem xx_add_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (u + v)) =
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h v) := by
  have hrel : GM.Rel c (GM.eqr (GM.xx l j k h u * GM.xx l j k h v) (GM.xx l j k h (u + v))) :=
    GM.Rel.Im1a l j k h u v
  have h_eq := mk_eqr c (GM.xx l j k h u * GM.xx l j k h v) (GM.xx l j k h (u + v)) hrel
  calc
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (u + v))
      = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u * GM.xx l j k h v) := by
        rw [← h_eq]
    _ = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *

```

```

    PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v) := by
      rw [map_mul]

theorem yy_add_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (u + v)) =
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h u) *
      PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) := by
have hrel : GM.Rel c (GM.eqr (GM.yy 1 j k h u * GM.yy 1 j k h v) (GM.yy 1 j k h (u + v))) :=
  GM.Rel.Im1b 1 j k h u v
have := mk_eqr c (GM.yy 1 j k h u * GM.yy 1 j k h v) (GM.yy 1 j k h (u + v)) hrel
simpa [GM.eqr, mul_inv_rev] using this.symm

theorem xx_sub_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (a b : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a - b)) =
    PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
      (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h b))-1 := by
have hzero : PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (0 : ℂ)) = 1 := by
have hrel : GM.Rel c (GM.eqr (GM.xx 1 j k h (0 : ℂ)) 1) := by
  have := GM.Rel.Im1a 1 j k h (0 : ℂ) (0 : ℂ)
  simpa [GM.eqr] using this
have hmem : GM.eqr (GM.xx 1 j k h (0 : ℂ)) 1 ∈ GM.Rels c := hrel
exact PresentedGroup.one_of_mem hmem
have hneg (u : ℂ) : PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (-u)) =
  (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u))-1 := by
have h := xx_add_mk c l j k h u (-u)

have hzero' : (u + (-u : ℂ)) = (0 : ℂ) := by ring
rw [hzero'] at h
rw [hzero] at h

exact eq_inv_of_mul_eq_one_right h.symm
calc
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a - b))
    = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a + (-b))) := by ring
  _ = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
    PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (-b)) := xx_add_mk c l j k h a (-b)
  _ = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
    (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h b))-1 := by rw [hneg b]

theorem yy_sub_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (a b : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) =
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h a) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
have hRel : GM.Rel c (GM.eqr (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) (GM.yy 1 j k h a)) := by
  have := GM.Rel.Im1b 1 j k h (a - b) b
  simpa [sub_add_cancel] using this
have hEq := mk_eqr c (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) (GM.yy 1 j k h a) hRel

have hMul : PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) =
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) *
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b) := by
  simp
  rw [hMul] at hEq

calc

```

```

PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b))
  = (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) *
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b)) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
  simp
_ = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h a) *
  (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
  rw [hEq]

theorem xx_conj_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^ (j - 1) * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
  have him6b := GM.Rel.Im6b 1 j k h s v
  have hmem : GM.eqr (GM.h2 s * GM.xx 1 j k h v * (GM.h2 s)-1) (GM.xx 1 j k h ((s : ℂ) ^ (j - 1) * v)) ∈
  ↪ GM.Rels c := him6b
  have heq := PresentedGroup.mk_eq_mk_of_mul_inv_mem hmem
  rw [← heq]
  simp [map_mul, map_inv, commutatorElement_def] using Subgroup.commutator_mem_commutator
  (g1 := PresentedGroup.mk (GM.Rels c) (GM.h2 s))
  (g2 := PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v))
  (h1 := show PresentedGroup.mk (GM.Rels c) (GM.h2 s) ∈ (T : Subgroup (GM.Grp c)) from Subgroup.mem_top _)
  (h2 := show PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v) ∈ (T : Subgroup (GM.Grp c)) from
  ↪ Subgroup.mem_top _)

open GM in
theorem yy_conj_mem_aiprover_s0 (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
  have h_rel := Rel.Im6d 1 j k h s v
  have h_eq := mk_eqr c (GM.h2 s * GM.yy 1 j k h v * (GM.h2 s)-1)
    (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) h_rel
  have h_mk : PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) =
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 := by
  calc
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) =
      PresentedGroup.mk (GM.Rels c) (GM.h2 s * GM.yy 1 j k h v * (GM.h2 s)-1) := h_eq.symm
    _ = (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
      (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 := by simp [map_mul]
  rw [h_mk]
  have h_comm : (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c)-1 =
    [[(PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c),
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c)]] := rfl
  rw [h_comm]
  dsimp [commutator]
  apply Subgroup.subset_closure
  exact ⟨(PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c), Subgroup.mem_top _,
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c), Subgroup.mem_top _, rfl⟩

theorem yy_conj_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
  first
  | exact yy_conj_mem_aiprover_s0

```

```

| (intros; apply yy_conj_mem_aiprover_s0 <=> assumption)

theorem xx_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) ∈ commutator (GM.Grp c) := by
  have hpos_or : j - 1 = 0 ∨ 0 < j - 1 := Nat.eq_zero_or_pos (j - 1)
  rcases hpos_or with (hjl | hdpos)
  · rcases h with ⟨hlt, _, _⟩
    have : 1 < 1 :=
      Nat.lt_of_lt_of_le hlt (Nat.le_of_sub_eq_zero hjl)
    exact absurd this (Nat.lt_irrefl _)
  · rcases exists_unit_pow_ne_one (j - 1) hdpos with ⟨s, hs⟩
    have hden_ne_zero : (s : ℂ) ^ (j - 1) - 1 ≠ 0 := by
      intro hzero
      have h_one : (s : ℂ) ^ (j - 1) = 1 := sub_eq_zero.mp hzero
      exact hs h_one
    set v := u / ((s : ℂ) ^ (j - 1) - 1) with hv_def
    have h_eq : u = ((s : ℂ) ^ (j - 1) - 1) * v := by
      dsimp [v]
      field_simp [hden_ne_zero]
    rw [h_eq]
    have : ((s : ℂ) ^ (j - 1) - 1) * v = ((s : ℂ) ^ (j - 1)) * v - v := by ring
    rw [this]
    rw [xx_sub_mk c l j k h ((s : ℂ) ^ (j - 1)) * v] v]
    exact xx_conj_mem c l j k h s v

theorem yy_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy l j k h u) ∈ commutator (GM.Grp c) := by
  have hjl : 0 < j - 1 := by
    have := h.1
  omega
  obtain ⟨s, hs⟩ := exists_unit_pow_ne_one (j - 1) hjl
  have ht : ((s : ℂ) ^ (j - 1))-1 - 1 ≠ 0 := by
    intro h0
    apply hs
  have h1 : ((s : ℂ) ^ (j - 1))-1 = 1 := sub_eq_zero.mp h0
  first
    | exact inv_eq_one.mp h1
    | simpa using h1
  have hu : ((s : ℂ) ^ (j - 1))-1 * (u / (((s : ℂ) ^ (j - 1))-1 - 1)) -
    u / (((s : ℂ) ^ (j - 1))-1 - 1) = u := by
    rw [← sub_one_mul]
  first
    | exact mul_div_cancel0 u ht
    | (rw [mul_comm]; exact div_mul_cancel0 u ht)
    | field_simp
  have key := yy_conj_mem c l j k h s (u / (((s : ℂ) ^ (j - 1))-1 - 1))
  first
    | (rw [← yy_sub_mk c l j k h, hu] at key; exact key)
    | (rw [← hu, yy_sub_mk c l j k h]; exact key)

theorem ww_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.ww l j k h s) ∈ commutator (GM.Grp c) := by
  let f : GM.F c →* GM.Grp c := PresentedGroup.mk (GM.Rels c)
  have hxx : f (GM.xx l j k h (s : ℂ)) ∈ commutator (GM.Grp c) := by
    dsimp [f]
    simpa using xx_mem c l j k h (s : ℂ)
  have hyy : f (GM.yy l j k h (-(s : ℂ)-1) / GM.cc l j) ∈ commutator (GM.Grp c) := by
    simpa using yy_mem c l j k h (-(s : ℂ)-1) / GM.cc l j)

```

```

have hprod : f (GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * GM.xx 1 j k h (s : ℂ)
↪ ∈ commutator (GM.Grp c) := by
  have hcalc : f (GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * GM.xx 1 j k h (s :
↪ ℂ)) =
    f (GM.xx 1 j k h (s : ℂ)) * f (GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * f (GM.xx 1 j k h (s :
↪ ℂ)) := by
    simp [map_mul]
    rw [hcalc]
    exact Subgroup.mul_mem _ (Subgroup.mul_mem _ hxx hyy) hxx

have hww : GM.ww 1 j k h s = GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1) / GM.cc 1 j) * GM.xx 1 j
↪ k h (s : ℂ) := rfl
rw [hww]
exact hprod

theorem h1h2_neg_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (-s) * GM.h2 (-(s-1))) ∈ commutator (GM.Grp c) := by
  have hrel : GM.Rel c (GM.eqr (GM.wm s * GM.wm 1) (GM.h1 (-s) * GM.h2 (-(s-1)))) :=
    GM.Rel.Re3 s
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.wm s * GM.wm 1) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 (-s) * GM.h2 (-(s-1))) :=
    mk_eqr c (GM.wm s * GM.wm 1) (GM.h1 (-s) * GM.h2 (-(s-1))) hrel
  have h_wm_s : PresentedGroup.mk (GM.Rels c) (GM.wm s) ∈ commutator (GM.Grp c) :=
    wm_mem c hc s
  have h_wm_1 : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) :=
    wm_mem c hc 1
  have h_prod : PresentedGroup.mk (GM.Rels c) (GM.wm s * GM.wm 1) ∈ commutator (GM.Grp c) :=
    Subgroup.mul_mem _ h_wm_s h_wm_1
  rw [← h_eq]
  exact h_prod

theorem h1h2_inv_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (x : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 x * GM.h2 x-1) ∈ commutator (GM.Grp c) := by
  have hrel := GM.Rel.Re3 (c := c) (-x)
  have h_simplified : (GM.h1 (c := c) (-(-x)) * GM.h2 (c := c) (-(x-1))) = (GM.h1 (c := c) x * GM.h2 (c
↪ := c) x-1) := by
    simp
  have hmem : PresentedGroup.mk (GM.Rels c) (GM.wm (-x) * GM.wm 1) ∈ commutator (GM.Grp c) := by
    have h1 : PresentedGroup.mk (GM.Rels c) (GM.wm (-x)) ∈ commutator (GM.Grp c) := wm_mem c hc (-x)
    have h2 : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) := wm_mem c hc 1
    exact Subgroup.mul_mem _ h1 h2
  have heq := mk_eqr c (GM.wm (-x) * GM.wm 1) (GM.h1 x * GM.h2 x-1) (by
    simpa [h_simplified] using hrel)
  rw [← heq]
  exact hmem

theorem im3_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c 1 j k) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))
  ∈ commutator (GM.Grp c) := by
  set w1 := GM.ww 1 j k h (s ^ ((1 + 1) * (j - 1))) with hw1
  set w0 := GM.ww 1 j k h 1 with hw0
  have hIm3_rel : GM.Rel c (GM.eqr (w1 * w0) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))) :=
    GM.Rel.Im3 1 j k h s
  have h_mul_inv : w1 * w0 * (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1)))-1 ∈ GM.Rels c := by
    simpa [GM.Rels] using hIm3_rel
  have h_mem' : PresentedGroup.mk (GM.Rels c) (GM.eqr (w1 * w0) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 +
↪ 1)))) = 1 :=
    PresentedGroup.one_of_mem hIm3_rel
  have h_eq : PresentedGroup.mk (GM.Rels c) (w1 * w0) =

```

```

    PresentedGroup.mk (GM.Rels c) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (1 + 1))) := by
  apply PresentedGroup.mk_eq_mk_of_mul_inv_mem h_mul_inv
have h_prod_mem : PresentedGroup.mk (GM.Rels c) (w1 * w0) ∈ commutator (GM.Grp c) := by
  have h1 : PresentedGroup.mk (GM.Rels c) w1 ∈ commutator (GM.Grp c) :=
    ww_mem c 1 j k h (s ^ ((1 + 1) * (j - 1)))
  have h2 : PresentedGroup.mk (GM.Rels c) w0 ∈ commutator (GM.Grp c) :=
    ww_mem c 1 j k h 1
  exact Subgroup.mul_mem (commutator (GM.Grp c)) h1 h2
rw [← h_eq]
exact h_prod_mem

theorem h1sq_h2_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (y : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (y ^ 2) * GM.h2 y) ∈ commutator (GM.Grp c) := by
have hvalid : GM.Valid c 0 2 1 := ⟨by decide, by decide, hc⟩
have hw1 := ww_mem c 0 2 1 hvalid (y ^ 2)
have hw2 := ww_mem c 0 2 1 hvalid 1
have hprod : PresentedGroup.mk (GM.Rels c)
  (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1) ∈ commutator (GM.Grp c) :=
  (commutator (GM.Grp c)).mul_mem hw1 hw2
have him3 : GM.Rel c (GM.eqr (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1)
  (GM.h1 (y ^ 2) * GM.h2 y)) := by
  have := GM.Rel.Im3 0 2 1 hvalid (-y)
  simpa [show ((-y : ℂ×) : ℂ) ^ ((0+1)*(2-0)) = (y : ℂ)^2 by norm_num] using this
have heq : PresentedGroup.mk (GM.Rels c)
  (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1) =
  PresentedGroup.mk (GM.Rels c) (GM.h1 (y ^ 2) * GM.h2 y) := by
  simpa [GM.Rels] using PresentedGroup.mk_eq_mk_of_mul_inv_mem him3
rw [← heq]
exact hprod

theorem h1_cube_eq (c : ℕ → ℕ) (z : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (z ^ 3)) =
  PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 z-1) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 z-1))-1 := by
let φ := PresentedGroup.mk (GM.Rels c)
have hφ_mul : ∀ x y : GM.F c, φ (x * y) = φ x * φ y := by
  intro x y; exact MonoidHom.map_mul φ x y
have hφ_inv : ∀ x : GM.F c, φ (x-1) = (φ x)-1 := by
  intro x; exact MonoidHom.map_inv φ x

have h1_mul : ∀ s t : ℂ×, φ (GM.h1 (c := c) (s * t)) = φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) t) := by
  intro s t
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) t) (GM.h1 (c := c) (s * t))) :=
    GM.Rel.H1a (c := c) s t
  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)

  have hcalc : φ (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) t) (GM.h1 (c := c) (s * t))) =
    φ (GM.h1 (c := c) s * GM.h1 (c := c) t) * (φ (GM.h1 (c := c) (s * t)))-1 := by
    simp [GM.eqr, hφ_mul, hφ_inv]
  rw [hcalc] at h_one

have h_eq : φ (GM.h1 (c := c) s * GM.h1 (c := c) t) = φ (GM.h1 (c := c) (s * t)) := by

```

```

    calc
       $\varphi (GM.h1 (c := c) s * GM.h1 (c := c) t) =$ 
       $(\varphi (GM.h1 (c := c) s * GM.h1 (c := c) t) * (\varphi (GM.h1 (c := c) (s * t)))^{-1}) *$ 
       $\varphi (GM.h1 (c := c) (s * t)) := \text{by group}$ 
       $\_ = 1 * \varphi (GM.h1 (c := c) (s * t)) := \text{by rw [h\_one]}$ 
       $\_ = \varphi (GM.h1 (c := c) (s * t)) := \text{by simp}$ 
    rw [h $\varphi$ _mul] at h_eq
    exact h_eq.symm

have h1_one :  $\varphi (GM.h1 (c := c) 1) = 1 := \text{by}$ 
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) (GM.h1 (c := c) (1 * 1))) :=
    GM.Rel.H1a (c := c) 1 1
  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)

have hcalc :  $\varphi (GM.eqr (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) (GM.h1 (c := c) (1 * 1))) =$ 
   $\varphi (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) * (\varphi (GM.h1 (c := c) (1 * 1)))^{-1} := \text{by}$ 
  simp [GM.eqr, h $\varphi$ _mul]
  rw [hcalc] at h_one

rw [mul_one] at h_one

rw [h $\varphi$ _mul] at h_one

calc
   $\varphi (GM.h1 (c := c) 1) = (\varphi (GM.h1 (c := c) 1) * \varphi (GM.h1 (c := c) 1)) * (\varphi (GM.h1 (c := c) 1))^{-1} := \text{by}$ 
 $\hookrightarrow \text{group}$ 
   $\_ = 1 := h\_one$ 

have h1_inv :  $\forall s : \mathbb{C}^\times, \varphi (GM.h1 (c := c) s^{-1}) = (\varphi (GM.h1 (c := c) s))^{-1} := \text{by}$ 
  intro s
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) s^{-1}) (GM.h1 (c := c) (s * s^{-1}))) :=
    GM.Rel.H1a (c := c) s s^{-1}

  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)
  have hcalc :  $\varphi (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) s^{-1}) (GM.h1 (c := c) (s * s^{-1}))) =$ 
     $\varphi (GM.h1 (c := c) s * GM.h1 (c := c) s^{-1}) * (\varphi (GM.h1 (c := c) (s * s^{-1})))^{-1} := \text{by}$ 
    simp [GM.eqr, h $\varphi$ _mul, h $\varphi$ _inv]
  rw [hcalc] at h_one

have h_s_one :  $\varphi (GM.h1 (c := c) (s * s^{-1})) = 1 := \text{by}$ 
  have :  $s * s^{-1} = (1 : \mathbb{C}^\times) := \text{by simp}$ 
  rw [this]
  exact h1_one
  rw [h_s_one] at h_one

have :  $(1 : \text{PresentedGroup } (GM.Rels c))^{-1} = 1 := \text{by simp}$ 
  rw [this, mul_one] at h_one

calc
   $\varphi (GM.h1 (c := c) s^{-1}) = 1 * \varphi (GM.h1 (c := c) s^{-1}) := \text{by simp}$ 

```

```

_ = ((φ (GM.h1 (c := c) s))-1 * φ (GM.h1 (c := c) s)) * φ (GM.h1 (c := c) s-1) := by simp
_ = (φ (GM.h1 (c := c) s))-1 * (φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) s-1)) := by group
_ = (φ (GM.h1 (c := c) s))-1 * 1 := by
  rw [show φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) s-1) = 1 from h_one]
_ = (φ (GM.h1 (c := c) s))-1 := by simp

have h_pow_h1 : ∀ (x : ℂ×) (n : ℕ), φ (GM.h1 (c := c) (x ^ n)) = (φ (GM.h1 (c := c) x)) ^ n := by
  intro x n
  induction' n with k ih
  ·
  · simp [h1_one]
  ·
  rw [pow_succ, pow_succ]

  rw [h1_mul (x ^ k) x]
  rw [ih]

have h_inv_h1 : ∀ (x : ℂ×), φ (GM.h1 (c := c) x-1) = (φ (GM.h1 (c := c) x))-1 := h1_inv

calc
  φ (GM.h1 (c := c) (z ^ 3)) = (φ (GM.h1 (c := c) z)) ^ 3 := by rw [h_pow_h1]
  _ = (φ (GM.h1 (c := c) z)) * ((φ (GM.h1 (c := c) z))-1 ^ 2)-1 := by group
  _ = (φ (GM.h1 (c := c) z) * φ (GM.h2 (c := c) z-1)) * (((φ (GM.h1 (c := c) z))-1 ^ 2) * φ (GM.h2 (c :=
→ c) z-1))-1 := by group
  _ = (φ (GM.h1 (c := c) z) * φ (GM.h2 (c := c) z-1)) * ((φ (GM.h1 (c := c) (z-1 ^ 2)) * φ (GM.h2 (c :=
→ c) z-1))-1) := by
  simp [h_pow_h1, h_inv_h1]
  _ = PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 z-1) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1 ^ 2) * GM.h2 z-1))-1 := by
  simp [φ, hφ_mul]

theorem exists_unit_cube (s : ℂ×) : ∃ z : ℂ×, z ^ 3 = s := by
  obtain ⟨w, hw⟩ := IsAlgClosed.exists_pow_nat_eq (s : ℂ) (by decide : 0 < 3)
  have hw_ne_zero : w ≠ 0 := by
    intro hzero
    rw [hzero, zero_pow (by decide : 3 ≠ 0)] at hw
  have hs_ne_zero : (s : ℂ) ≠ 0 := Units.ne_zero s
  exact hs_ne_zero (Eq.symm hw)
  let z : ℂ× := Units.mk0 w hw_ne_zero
  refine ⟨z, ?_⟩
  apply Units.ext_iff.mpr
  calc
    (z : ℂ) ^ 3 = w ^ 3 := rfl
    _ = (s : ℂ) := hw

theorem h1_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s) ∈ commutator (GM.Grp c) := by
  rcases exists_unit_cube s with ⟨z, hz⟩

  have h_comm_z : PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 (z-1)) ∈ commutator (GM.Grp c) :=
    h1h2_inv_mem c hc z

```

```

have h_comm_z_inv : PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 (z-1)) ∈
  commutator (GM.Grp c) :=
  h1sq_h2_mem c hc (z-1)

have h_cube := h1_cube_eq c z

have h_prod : PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 (z-1)) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 (z-1)))-1 ∈
  commutator (GM.Grp c) := by
  apply Subgroup.mul_mem
  · exact h_comm_z
  · apply Subgroup.inv_mem
    exact h_comm_z_inv

have h_cube_mem : PresentedGroup.mk (GM.Rels c) (GM.h1 (z ^ 3)) ∈ commutator (GM.Grp c) := by
  rw [h_cube]
  exact h_prod

simp [hz] using h_cube_mem

theorem h2_eq_conj (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) =
  PresentedGroup.mk (GM.Rels c) (GM.wm 1) * PresentedGroup.mk (GM.Rels c) (GM.h1 s) *
  (PresentedGroup.mk (GM.Rels c) (GM.wm 1))-1 := by

have h_eq := mk_eqr c (GM.wm 1 * GM.h1 s * (GM.wm 1)-1) (GM.h2 s) (by
  apply GM.Rel.Re5a)

simp [PresentedGroup.mk, map_mul, map_inv] using h_eq.symm

theorem h2_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) ∈ commutator (GM.Grp c) := by
have hnormal : (commutator (GM.Grp c)).Normal := inferInstance
have hh1 : PresentedGroup.mk (GM.Rels c) (GM.h1 s) ∈ commutator (GM.Grp c) := h1_mem c hc s
have hww : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) := wm_mem c hc 1
rw [h2_eq_conj c s]
exact hnormal.conj_mem (PresentedGroup.mk (GM.Rels c) (GM.h1 s)) hh1
  (PresentedGroup.mk (GM.Rels c) (GM.wm 1))

theorem solution (c : ℕ → ℕ) (hc : 1 ≤ c 2) :
  commutator (GM.Grp c) = ⊤ := by
  apply subgroup_top_of_gens
  intro g
  cases g with
  | H1 s => exact h1_mem c hc s
  | H2 s => exact h2_mem c hc s
  | Xm u => exact xm_mem c hc u
  | Ym u => exact ym_mem c hc u
  | X l j k h u => exact xx_mem c l j k h u
  | Y l j k h u => exact yy_mem c l j k h u

```